Meditation: Agent Harness Orchestration Patterns

Full Depth Synthesis Report

Generated: 2026-05-09 | 39 exploration files across 3 recursion levels
Repository: CRUX-Compress | Topic: Agent harness orchestration patterns

Contents

1. Executive Summary

Overview

Three branches explored the orchestration of multi-agent systems through complementary lenses: how agents share data (state coordination), what happens when things fail (resilience), and how the system stays bounded (resource governance). Across 39 exploration files spanning 3 recursion levels, a unified architectural model emerged — one that is already partially implemented in the CRUX-Compress codebase and points toward concrete improvements.

Branch 1: State Coordination and Handoff Mechanisms

Key Discoveries

State coordination operates as a **three-layer protocol stack**:

Layer	Concern	Mechanisms
Transport	Where data moves	File-based polling, predictable paths, existence as signal
Propagation	How intent flows	`alwaysApply` rules, spawn-time args, config pull
Serialization	What is exchanged	Frontmatter contracts, four-format stack (JSON → YAML → Markdown → CRUX)

A **push-pull duality** unifies all three layers: pull (agents independently read shared state from known locations) for persistent defaults, push (explicit parameters at spawn time) for ephemeral overrides. This mirrors the plugin system's advisory-gate-with-opt-out architecture.

The system consistently chooses **explicit over implicit** (source fields, alwaysApply rules, frontmatter contracts over inference from paths or timestamps), **repair over prevention** (eventual consistency with REM sleep rather than strict referential integrity), and **consumer-optimized formats** (LLM comprehension as the primary design driver).

Central Challenge: The Knows-vs-Acts Gap

Delivering state to an agent is necessary but not sufficient for behavioral compliance. This manifests as: - Detection without verification in file protocols - Loaded but ignored rules in session propagation - Defined but violated contracts in serialization

The proposed **compliance testing pyramid** — directive trust (weakest) → self-report → output-pattern analysis → side-effect observation → adversarial verification (strongest) — provides a framework for systematically closing this gap.

Branch 2: Failure Handling and Resilience

Key Discoveries

The harness implements a **four-layer defence-in-depth stack**:

Layer	Function	Example
Demand reduction	Eliminate unnecessary work	Shared-agent-runs (N calls → 1), skip-by-default
Concurrency limiting	Reduce peak pressure	maxForks: 2, dependency-based phasing
Reactive recovery	Recover from transient failures	Exponential backoff + jitter (BASE=2s, MAX=60s, 5 retries)
Hard backstop	Kill runaway processes	Wall-clock deadline (60min default)

The deepest epistemological finding: **an agent's narrative is a hypothesis, the filesystem is ground truth**. Adversarial verification catches real failures at a 25% hit rate, with variance correlating to integration complexity rather than task count.

Three Critical Gaps

1. **Semantic hang detection**: Five undetected hang patterns (tool-call loops, output stagnation, thinking-without-acting, file-polling deadlocks, phantom completions). A **progress token framework** — tracking `lastProgressTimestamp` based on novel actions — would unify all five into a single configurable mechanism.
2. **Partial-result degradation**: Current all-or-nothing architecture wastes successful work. For exploratory operations (meditation, recall), graceful consolidation with gap annotation is strictly better. For transactional operations (spec execution, memory creation), correlation-first triage should distinguish systemic from isolated failures.
3. **Graduated verification**: Uniform adversarial verification is wasteful. A risk-scoring heuristic (file creation +3, cross-cutting registration +3, documentation with paths +2, first-time agent +2) should drive tiered verification intensity.

Branch 3: Resource Governance and Bounded Execution

Key Discoveries

Resource governance follows a **three-layer control architecture** analogous to TCP congestion control:

Layer	Function	Agent Equivalent	TCP Analogue
Demand reduction	Eliminate unnecessary work	Batching, skip-by-default, specialization	Connection reuse
Admission control	Bound concurrent execution	Static caps, phase parallelism, deadlines	Congestion window
Reactive recovery	Handle residual transients	Exponential backoff, jitter, retry budgets	Fast retransmit

Three unifying principles: **reduce demand before increasing tolerance** (eliminating work compounds savings), **static ceilings with negotiable compliance paths** (maximum cost is predictable, path to compliance is flexible), and **human escalation as universal terminal** (agents refuse to make irreversible decisions).

Central Insight: Cost and Blast Radius Are Orthogonal

A 13-agent recursive meditation is expensive but safe (read-only). A single-agent destructive operation is cheap but dangerous. Both dimensions must independently escalate governance tier classification. The meditate system's read-only-with-opt-in-persistence pattern demonstrates that bounding blast radius is a governance concern distinct from bounding cost.

The Missing Middle Layer

The codebase has strong implementations of layers 1 (demand reduction) and 3 (reactive recovery) but nearly no layer-2 admission control beyond static caps. Missing: dynamic concurrency controllers, admission queuing, staggered starts, correlated failure detection, and graceful shutdown signals for hard deadlines.

Cross-Branch Connections

1. The Defence-in-Depth Stack Is Universal

All three branches independently converged on layered architectures with the same ordering: **eliminate** → **bound** → **recover** → **escalate**. This isn't coincidence — it reflects the fundamental principle that reducing demand has compounding returns (fewer calls = lower cost AND fewer failures AND less state to coordinate), while increasing tolerance has diminishing returns.

2. The Push-Pull Duality Spans All Concerns

Concern	Pull (shared state)	Push (explicit params)
Coordination	Config files, alwaysApply rules	Spawn-time arguments
Resilience	Filesystem ground truth	Error context propagation
Governance	Static caps in config	Session-scope flag inheritance

The mature pattern is hybrid: pull for defaults, push for overrides. This holds across all three domains.

3. File-Based Coordination Enables Partial-Result Recovery

The meditation protocol's file-based coordination is simultaneously a coordination mechanism (Branch 1), a resilience enabler (Branch 2), and a governance primitive (Branch 3): - **Coordination**: Predictable paths enable decoupled agents to find each other's outputs - **Resilience**: Persistent child outputs survive parent crashes, enabling selective retry at the lowest level - **Governance**: Read-only exploration bounds blast radius; file existence gates persistence decisions

4. The Knows-vs-Acts Gap and Trust Inversion Are Two Faces of One Problem

Branch 1's "knows-vs-acts gap" (delivering state ≠ compliance) and Branch 2's "trust inversion" (agent narratives are hypotheses, filesystem is ground truth) describe the same fundamental challenge: **LLM agents are probabilistic executors operating in deterministic environments**. The system must bridge this gap at every layer — through redundant constraint expression, adversarial verification, and ground-truth observation rather than self-report.

5. Escalation Ladders Are Composable Primitives

Branch 3's formal escalation ladder (initial → step → ceiling → invariant → terminal) appears across all branches: - **Coordination**: Schema evolution is an additive escalation ladder (optional → required → canonical) - **Resilience**: Retry strategies are time-domain escalation ladders (2s → 4s → 8s → 16s → 32s → escalate) - **Governance**: Compression targets are space-domain escalation ladders (33% → aggressive → maximum → manual review)

This five-parameter interface (initial level, step function, ceiling, invariant predicate, terminal action) could be abstracted into a reusable primitive.

Potential Directions for Further Exploration

1. **Progress Token Framework**: Design and prototype a `lastProgressTimestamp` tracker that detects all five semantic hang patterns through a single mechanism. This is the highest-value concrete improvement — a single hung leaf currently blocks entire meditation trees indefinitely.
2. **Dynamic Admission Control**: Prototype a TCP-inspired admission controller that adjusts concurrency based on rate-limit signal frequency. The static-cap gap is the largest architectural hole across all three branches.
3. **Compliance Testing Infrastructure**: Move critical coordination checks from levels 1-3 (directive trust, self-report, output patterns) to levels 4-5 (side-effect observation, adversarial verification) of the compliance pyramid. Start with file-write verification — the most concrete and frequently-failed check.
4. **Escalation Ladder Abstraction**: Extract the five-parameter escalation ladder into a reusable skill or utility, replacing the scattered implementations in compression, retry, and type demotion.
5. **Cost-Blast-Radius Matrix**: Formalize the two-axis governance classification so every agent operation is tagged with both its cost tier and its blast radius tier, enabling independent gating.

Actionable Insights

Highest Priority (Concrete Bugs / Gaps)

- **Add polling timeout to meditate file-based coordination** — a single hung depth-3 agent currently blocks the entire tree forever
- **Widen jitter from ±15% to ±50%** (or decorrelated jitter) to safely support higher parallelism without thundering-herd collisions
- **Verify file writes after creation** — the Write tool can silently fail; `ls` after every critical write is cheap insurance

Medium Priority (Architectural Improvements)

- **Implement graceful partial-result consolidation** for exploratory operations — proceed with N-1 results and annotate the gap, rather than blocking on all N
- **Centralize path conventions** — five file-based coordination families with different naming conventions are specified in prose; a shared path-computation utility would prevent drift
- **Add graduated adversarial verification** with risk scoring to optimize the 25% hit rate against verification cost

Strategic (Design Patterns to Adopt)

- **Reduce demand before increasing tolerance** — this principle should be the default response to any resource pressure, before tuning retries or raising limits
- **Separate cost governance from blast-radius governance** — read-only exploration with opt-in persistence is the model for bounded-risk agent operations
- **Use retry frequency as a diagnostic signal** — if retries fire often, the proactive settings (not the retry parameters) need adjustment

2. Exploration Facets

Parent Context

Exploring agent harness orchestration patterns — how multi-agent systems coordinate execution, manage state handoff, handle failures, and bound recursion in LLM-based tooling environments like Cursor.

Facet Partitioning

The topic "agent harness orchestration patterns" naturally decomposes into three complementary concerns: the mechanisms by which agents share data and coordinate (the plumbing), the strategies for coping when parts of the system fail (the resilience), and the constraints that keep the system efficient and bounded (the governance). These three dimensions are orthogonal — each can be explored deeply without needing the others, yet together they fully characterize what makes an agent harness work.

Facets

Facet 1: State Coordination and Handoff Mechanisms

How do multi-agent systems pass state, context, and results between parent and child agents? This encompasses file-based coordination protocols (predictable paths, polling for existence, frontmatter contracts), message-based approaches (in-context returns, transcript parsing), session-scope inheritance (flags, config propagation to subagents), and the serialization formats that make inter-agent data exchange reliable.

Facet 2: Failure Handling and Resilience in Multi-Agent Workflows

How do agent harnesses detect, recover from, and adapt to partial failures when one or more agents in a parallel or sequential workflow crash, hang, produce malformed output, or hit rate limits? This covers retry strategies (exponential backoff, jitter), timeout and hang detection, graceful degradation when a branch fails, the boundary between automatic recovery and user escalation, and adversarial verification as a failure-catching mechanism.

Facet 3: Resource Governance and Bounded Execution

How do agent harnesses manage computational resources, control costs, and prevent unbounded growth? This includes recursion depth limits, concurrency caps (fan-out width, max parallel agents), cost-aware execution gating (skip-by-default for expensive operations), wall-clock deadlines, and the design patterns that trade off exploration breadth against resource consumption in LLM-powered multi-agent systems.

---

3. Branch 1: State Coordination and Handoff Mechanisms

Branch 1 — Depth 1 Synthesis

Subfocus Rationale

State coordination is the connective tissue of multi-agent orchestration. Without reliable mechanisms for passing context, results, and behavioral contracts between agents, every other orchestration concern — failure handling, resource governance, task decomposition — operates on unstable ground. This branch examines the "plumbing" that makes multi-agent systems coherent: how agents exchange data, propagate intent, and maintain contract fidelity across spawn boundaries.

Discoveries

From Memory Queries

Twelve memories were queried, spanning core principles, learned patterns, and documented failure modes:

- **ba74013** (learning, strength 2): Session-scope flags need explicit `alwaysApply` rules for subagent inheritance. Every session-flag spec must answer three questions: what inherits, what breaks inheritance, where is the contract documented.
- **49303e0** (redflag, strength 1): File writes can silently fail — the Write tool can return without persisting. Agent narratives are hypotheses; filesystem state is ground truth.
- **f8bd856** (learning, strength 1): When multiple writers contribute to a shared store, tag every entry with an explicit `source` field for provenance. Constants like `"adhoc"` are signals, not absence.
- **d944d7c** (redflag, strength 1): Multi-layer state (spec index + subtask files) drifts. The more local/specific layer is authoritative.
- **d5e503c** (learning, strength 1): Ordering of phase blocks in serialized rules encodes override precedence — exploiting LLM sequential attention.
- **31fec9d** (core, strength 1): Meditate enforces read-only exploration with a single persistence gate. Safety comes from separating exploration from side effects.
- **ba92c4e** (learning, strength 1): Sourcing interactive options from config keys (single-source-of-truth) prevents drift between the semantic model and its UI representations.
- **efc4c24** (learning, strength 1): Dependency-based phasing with parallel subagents eliminates coordination overhead for independent subtasks.
- **f8bdc0d** (core, strength 1): Agent files orchestrate; skill files contain detailed logic. This separation affects how state flows between architectural layers.
- **cd0c954** (core, strength 1): Archive original source files before overwriting — a write-safety primitive for data integrity.
- **96a7410** (redflag, strength 1): Tooling defaults must align with specification defaults — drift between implementations is silent until caught by dedicated audit.
- **3bf625d** (redflag, strength 1): Synthesis must not hallucinate connections — distinguish recalled vs discovered vs inferred provenance.

Cross-Memory Patterns

Three meta-patterns emerged from the memory corpus:

1. **Explicit over implicit**: The system consistently favors explicit state declarations (source fields, alwaysApply rules, frontmatter contracts) over inference from paths, timestamps, or context. Every memory touching coordination encodes this preference.

2. **Repair over prevention**: Rather than enforcing referential integrity at write-time (foreign keys, transactions), the system uses eventual consistency with periodic repair cycles (REM sleep, index rebuilds, validation passes). This matches the reality of crash-prone LLM agents.
3. **Layered redundancy**: Critical constraints are stated in multiple forms (declarative schema, procedural steps, proscriptive anti-patterns) to compensate for the probabilistic nature of LLM compliance.

Connections

The Three Coordination Layers Form a Complete Protocol Stack

The three child subfocuses revealed that state coordination operates as a protocol stack:

```
Layer 3: Serialization (data contracts)  — WHAT is exchanged
Layer 2: Propagation (session scope)     — HOW intent flows
Layer 1: Transport (file-based protocols) — WHERE data moves
```

Each layer makes independent design decisions, but they compose: frontmatter contracts (Layer 3) define what fields a file must contain, propagation rules (Layer 2) determine which agents load which behavioral directives, and file-based protocols (Layer 1) define the naming, detection, and verification mechanisms for the physical files. A failure at any layer cascades: a silently failed file write (Layer 1) renders a perfect frontmatter contract (Layer 3) irrelevant.

The Push-Pull Duality Spans All Three Layers

A unifying framework emerged: **pull** (each agent independently reads shared state from known locations) vs **push** (state is explicitly passed at spawn time).

Layer	Pull Mechanism	Push Mechanism
Transport	File polling (child writes, parent polls)	Direct parameter passing at spawn
Propagation	`alwaysApply` rules (IDE injects automatically)	Spawn-time arguments (ephemeral context)
Serialization	Config file read by all consumers	Override fields in spawn prompts

Pull achieves zero-coordination consensus for persistent state. Push is required for ephemeral state. The mature pattern is hybrid: pull for defaults, push for overrides — mirroring the plugin system's advisory gates with opt-out overrides (b0c02ea).

The Knows-vs-Acts Gap Is the Fundamental Challenge

The deepest insight across all three subfocuses: **delivering state to an agent is necessary but not sufficient for compliance**. This manifests differently at each layer:

- **Transport**: A file exists (detection) but may be invalid (verification gap)
- **Propagation**: A rule is loaded (knows) but may be ignored (acts gap) — competing MUST directives, attention dynamics, conditional parsing
- **Serialization**: A contract is defined (schema) but may be violated (enforcement gap) — eventual consistency rather than strict enforcement

The system's answer is a **compliance testing pyramid**: directive trust (weakest) → self-report → output-pattern analysis → side-effect observation → adversarial verification (strongest). Currently, most propagation checking operates at levels 1-3; advancing critical checks to levels 4-5 would close the gap.

File-Based Coordination Is a Mature but Informally Specified Protocol

Five distinct coordination families coexist (meditation positional encoding, dream summaries, memory files, sentinel files, spec state files), each with different naming conventions, detection strategies, and verification expectations. The protocol works because of three properties:

1. **Path determinism**: Any agent independently computes the same file path given the same inputs — no central broker needed
2. **Existence as signal**: File presence/absence is the primary coordination primitive, augmented by structured content
3. **Verification as complement**: Post-write verification (ls → Read → git status) compensates for unreliable persistence

But the protocol is specified in prose, not code. A concrete drift instance was identified: meditation depth-3 file numbering shows global vs relative indexing ambiguity in the spec itself.

Serialization Optimizes for the Primary Consumer

Format choices reflect an optimization hierarchy: **LLM comprehension > human reviewability > git trackability > machine parsing strictness**. This produces a four-format stack:

- **JSON**: Strict machine config (`.crux/crux-memories.json`)
- **YAML frontmatter**: Diffable metadata (memory files, meditation outputs)
- **Markdown body**: LLM-interpretable content (no deserialization needed)
- **CRUX notation**: Token-constrained LLM contexts (5-10x compression)

The "no formal schema" choice is deliberate: the primary parser (an LLM) is robust to ambiguities that would break traditional parsers, and schema evolution is additive-only, absorbed without version numbers or migration scripts.

Provenance as a Six-Layer DAG

Explicit provenance tags create a complete lifecycle graph:

1. **Origin** (`source` field) — who created this
2. **Merge** (`consolidated_from`) — what was it made of
3. **Transition** (`promoted_from`) — what did it used to be

4. **Compression** (`sourceArchive`) — where is the original
5. **Usage** (`.refs.yml`) — who consumed it
6. **Generation** (`generated` timestamp) — when was this derived

This survives every mutation: file moves, compression, consolidation, promotion. Inference-based provenance degrades at each step; explicit provenance is durable.

Child Subfocuses

Sub-1: File-Based Coordination Protocols

How agents use the filesystem as a communication bus — naming conventions, detection strategies (polling vs events vs sentinel files), write verification, and path determinism as the foundation of decoupled coordination.

Rationale: File-based coordination is the dominant inter-agent pattern in this codebase, and understanding its mechanics is prerequisite to understanding the other two subfocuses.

Sub-2: Session-Scope State Propagation

How flags, configuration, and behavioral contracts implicitly flow from parent to child agents — alwaysApply rules as inheritance vehicles, config-driven pull propagation, override precedence, and the knows-vs-acts gap.

Rationale: While file-based coordination handles explicit data exchange, much agent behavior is governed by implicit state that must propagate reliably across spawn boundaries.

Sub-3: Serialization Formats and Data Contracts

How inter-agent data is structured — prose-defined schemas, additive evolution, provenance tagging, the four-format stack, and the tension between LLM comprehension and traditional parsing strictness.

Rationale: The reliability of both file-based coordination and session-scope propagation depends on the data contracts that define what well-formed inter-agent communication looks like.

Child Insights

From Sub-1 (File-Based Coordination Protocols)

File-based coordination operates across three sub-layers: **naming** (deterministic path conventions enabling decoupled address computation), **detection** (polling, events, or flag files matched to latency requirements), and **verification** (post-write checks ensuring persistence and integrity). The most important insight is that detection and verification are complementary halves currently treated as separate concerns. The meditate command detects (polls for file existence) but doesn't verify (no content validation, no timeout for missing children). Combining archive-before-write (cd0c954) with verify-after-write (49303e0) creates transactional semantics equivalent to write-ahead logging. Three actionable gaps: (1) meditate's polling loop needs a timeout, (2) scattered write-safety primitives should be unified into a reusable skill, (3) path conventions hardcoded in prose should be centralized.

From Sub-2 (Session-Scope State Propagation)

Session-scope propagation uses a push-pull architecture: **pull** (shared config files read independently by all agents for persistent state) and **push** (alwaysApply rules for behavioral contracts, spawn-time arguments for ephemeral state). The `alwaysApply` mechanism is isomorphic to Unix environment variable export — the sole reliable broadcast channel because the IDE guarantees injection. Its fragility lies in silent degradation: missing flags, stale exception lists, and misordered phase blocks break inheritance without error signals. The deepest finding is the **knows-vs-acts gap**: propagation mechanisms ensure agents load directives, but compliance depends on LLM attention dynamics and conditional parsing. The only reliable bridge is a **compliance testing pyramid** from directive trust (weakest) through self-report, output-pattern analysis, and side-effect observation to adversarial verification (strongest).

From Sub-3 (Serialization Formats and Data Contracts)

Serialization is built on four architectural decisions: (1) **prose-defined schemas with procedural enforcement** optimized for LLM consumption rather than compiler validation; (2) **additive-only schema evolution** via a required/optional field partition that has absorbed three capability waves without versioning; (3) **explicit six-layer provenance** where no lineage question requires inference; and (4) **a consumer-optimized format stack** trading parsing strictness for LLM comprehension. The system consistently chooses repair (REM sleep) over prevention (foreign keys), reflecting a world where the most important data consumer is probabilistic.

Summary

State coordination in this multi-agent system operates as a three-layer protocol stack — transport (file-based protocols), propagation (session-scope inheritance), and serialization (data contracts) — unified by a push-pull duality where pull achieves zero-coordination consensus for persistent state and push handles ephemeral context.

Five key architectural principles emerge:

1. **Explicit over implicit**: Source fields, alwaysApply rules, and frontmatter contracts are preferred over inference from paths or context
2. **Repair over prevention**: Eventual consistency with periodic cleanup (REM sleep, index rebuilds) rather than strict referential integrity
3. **Consumer-optimized formats**: LLM comprehension is the primary design driver, producing a four-format stack from JSON to CRUX
4. **Path determinism enables decoupling**: Agents independently compute file locations without coordination, making the filesystem a viable message bus
5. **Layered redundancy**: Critical constraints are stated in multiple forms to compensate for probabilistic LLM compliance

The central challenge is the knows-vs-acts gap: delivering state to an agent is necessary but not sufficient for behavioral compliance. This manifests as detection-without-verification in file protocols, loaded-but-ignored rules in propagation, and defined-but-violated contracts in serialization. The system's compliance testing pyramid — from directive trust through side-effect observation to adversarial verification — provides the framework for systematically closing this gap. Currently operating at levels 1-3 for most coordination concerns, advancing critical checks to levels 4-5 represents the highest-value improvement opportunity.

Branch 1, Depth 2, Sub-1: File-Based Coordination Protocols

Subfocus Rationale

Among the three dimensions of state coordination (file-based protocols, session-scope propagation, serialization formats), file-based coordination is the dominant inter-agent communication pattern in this codebase. The meditation command, dream extraction, spec execution, memory system hooks, and MCP watcher all use the filesystem as their primary coordination bus. Understanding the protocols that make this work — naming conventions, detection strategies, and persistence guarantees — is foundational to understanding how the entire agent harness holds together.

Discoveries

The filesystem as a communication bus

This codebase treats the filesystem as a shared message bus where agents are producers and consumers. Five distinct coordination families coexist:

1. **Meditation tree coordination** — positional encoding (`branch-{N}-depth-{D}-sub-{S}.md`) with fixed bookend files (`facets.md` , `consolidation.md`). Up to 39 output files plus bookends in a single meditation.
2. **Dream summaries** — config-sourced templates (`dream-{slug}-{yyyymmdd}.md` via `summaryPattern` in `crux-memories.json`).
3. **Memory files** — slug-based naming with extension-encoded type discrimination (`.memory.md` vs `.memory.crux.md`).
4. **Sentinel/flag files** — presence-as-signal (`.crux/pending-index-rebuild.json` , `.crux/pending-compression.json`) carrying structured payloads.
5. **Spec state files** — positional within the work directory (`_execution-state.yml`).

Three detection patterns across a latency spectrum

Pattern	Latency	Use Case	Mechanism
Watchdog + debounce	~1-2s	MCP index freshness	OS filesystem events via Python `watchdog` , 1s debounce timer
ls-polling	10-30s	Agent-to-agent coordination	Shell `ls` at intervals; file existence = only signal
Pending-flag + session check	Minutes-hours	Cross-session state	Hook writes flag; next session-start reads it

Each is correctly matched to its use case's latency tolerance.

The trust gap: agent narrative vs filesystem evidence

Memory 49303e0 [memory:Agent-reported file creation must be verified on disk] established that the Write tool can silently fail. An agent can produce a detailed work log claiming file creation — with path, contents, and verification narrative — without the file existing on disk. Three observed root causes: silent tool failure, canvas-only emission, and agent self-deception (narrating without invoking Write).

Scattered but complete write-safety primitives

The codebase contains all the building blocks for transactional writes, but they exist as scattered patterns rather than a unified primitive: - **Pre-write archival** (memory cd0c954): compression skill archives originals to dated directories before overwriting - **Staging directories**: `install.py` downloads to `tempfile.mkdtemp()`, verifies checksums, then installs from staging - **Three-layer verification**: ls → Read → git status (prescribed by memory 49303e0) - **Checksum verification**: `install.py` SHA256 verification against release manifests - **Idempotent writes**: CRUD skill generates deterministic IDs via `sha256(title)[:7]`

Connections

1. Detection and verification are complementary halves of the coordination protocol

Detection (polling or events) answers "did a file appear?" Verification (ls + Read + git status) answers "is the file valid?" Neither alone is sufficient. The meditate command has detection (ls-polling) but no verification — a parent reads the file without checking integrity. The MCP watcher's debounce accidentally provides a weak verification proxy (waiting 1s past the last event naturally waits past CREATE→CLOSE_WRITE), but this is emergent, not designed. A complete coordination protocol needs both halves explicitly.

2. Archive-before-write + verify-after-write creates transactional semantics

Combining the archival pattern (cd0c954) with the verification protocol (49303e0) produces a transactional write envelope: (1) archive original → abort if fails, (2) write new content → may silently fail, (3) verify → if fails, recover from archive, (4) clean up original → only after verified success. This is functionally equivalent to write-ahead logging in databases.

3. Path determinism enables decoupled coordination without a broker

Every path convention is deterministic — given the same inputs, any agent independently computes the same path. This is what eliminates the need for a central coordinator. The consumer constructs the expected path *before* the producer writes, which is what makes polling possible. But determinism is fragile: if the path template is defined in prose (meditation) rather than config (dream), silent drift between producer and consumer goes undetected.

4. A live path convention inconsistency illustrates the drift risk

The meditation working directory tree diagram shows depth-3 files with globally sequential sub-indices (sub-1 through sub-9), but the protocol description passes `subfocusIndex` as 1-3 per parent — implying per-parent-relative indexing. If implemented literally, three depth-2 agents would all write `branch-1-depth-3-sub-{1,2,3}.md`, colliding on the same filenames. This is exactly the class of drift that memory 96a7410 (tooling defaults diverging from spec) and memory d944d7c (spec index text contradicting subtask details) warn about.

5. Sentinel files are architecturally distinct from output files but blur in practice

Pure sentinels would be empty (presence = boolean). But this codebase's sentinel files carry structured payloads — file lists, timestamps, `needsRebuild` flags. They're hybrids: existence gates whether to act, content determines what to act on. This works but creates a fragility: corrupt JSON passes the existence check but fails the content read. The `crux-detect-memory-changes.py` hook handles this with try/except, but the pattern needs explicit documentation.

6. The meditate polling loop has no timeout — a deadlock waiting to happen

If a child agent crashes without writing its output file, the parent blocks indefinitely. There is no `max_wait_time`, no fallback-on-timeout (proceed with N-1 branches), and no incomplete-file detection. Memory 49303e0 documents why this matters: the Write tool can silently fail, meaning a child that "completed" may never produce its output file.

Child Subfocuses

Sub-1: Polling-for-Existence vs Event-Driven Notification

How do parent agents detect when child output files appear? Trade-offs between naive ls-polling (meditate), watchdog event handlers with debouncing (MCP watcher), and pending-flag files (cross-session hooks). Focuses on latency, resource usage, race conditions, and failure modes of each detection strategy.

Sub-2: Write Verification and Persistence Guarantees

Given that file writes can silently fail, what verification protocols ensure persistence? Covers the three-layer verification stack (ls → Read → git status), pre-write archival, staging directories, checksum verification, idempotent writes, and the gap between having these primitives scattered vs unified.

Sub-3: Path Convention Design and the Coordination Namespace

How do predictable naming patterns enable decoupled coordination? Covers the five convention families, what makes a convention good (determinism, collision avoidance, dual readability), sentinel vs output files as coordination primitives, config-sourced vs prose-hardcoded templates, and the risks of convention drift.

Child Insights

From Sub-1 (Polling vs Event-Driven)

The three detection patterns form a latency spectrum correctly matched to use cases: watchdog events (~1s) for live MCP queries, ls-polling (10-30s) for agent coordination where children take minutes, pending-flags (session-granularity) for cross-session state. The key insight is that **detection alone is insufficient** — file existence doesn't guarantee file completeness. The watchdog's 1-second debounce *accidentally* mitigates the write-completeness race (by waiting past CREATE→MODIFY→CLOSE_WRITE sequences), but polling has no equivalent safeguard. The most actionable gap: meditate's polling loop has no timeout or abort-on-missing-child, so a child crash silently deadlocks the parent. Platform-specific failure modes (inotify limits, NFS staleness) are unlikely concerns at current scale but would become relevant for org-wide automation ideas like memory 515c010.

From Sub-2 (Write Verification)

Write verification is the weakest link in file-based coordination. The codebase has scattered but complete building blocks: pre-write archival (cd0c954) creates rollback capability, staging directories (install.py) approximate atomic writes, the three-layer verification protocol (49303e0) catches different failure classes at different costs, and checksum verification provides bit-level integrity evidence. Combined, archive-before-write + verify-after-write creates transactional semantics equivalent to write-ahead logging. Adversarial verification (memory 6c16dc6, 25% hit rate) provides a fourth verification layer — because the writing agent may skip or misinterpret its own verification. The central gap: these primitives aren't unified into a formal `write-and-verify` skill that all agents invoke consistently. Each skill reimplements verification ad-hoc, inviting inconsistency.

From Sub-3 (Path Conventions)

Path conventions are the addressing layer that makes decoupled coordination possible — consumers independently compute expected locations without communicating with producers. Good conventions share four properties: determinism, collision avoidance, human readability, and machine parseability. The five convention families each optimise for their domain. The primary risk is convention drift — when templates are defined in one place and reconstructed in another. The codebase already has a live instance: the meditation depth-3 file numbering shows global vs relative indexing ambiguity. Centralising templates (as `summaryPattern` in config does) mitigates drift; hardcoding in prose invites it. The sentinel/output distinction is architecturally meaningful but blurry in practice — sentinel files carry structured payloads, making them hybrids that need explicit error handling for corrupt content.

Summary

File-based coordination in this codebase is a mature but informally specified protocol operating across three layers: **naming** (deterministic path conventions enabling decoupled address computation), **detection** (polling, events, or flag files matched to latency requirements), and **verification** (post-write checks ensuring persistence and integrity). The architecture works because these layers compose: agents compute where to write (naming), other agents detect when the write appears (detection), and either party can confirm the write is valid (verification).

The most important cross-cutting insight is that **detection and verification are complementary halves that are currently treated as separate concerns**. The meditate command detects but doesn't verify. The memory 49303e0 protocol verifies but is prescribed as a one-off practice rather than a composable primitive. Combining archive-before-write with verify-after-write creates transactional semantics, but this pattern isn't codified as a reusable skill.

Three actionable gaps emerge: (1) meditate's polling loop needs a timeout and graceful degradation for missing children, (2) the scattered write-safety primitives should be unified into a formal write-and-verify skill, and (3) path conventions hardcoded in prose (especially meditation's depth-3 numbering scheme) should be centralised to prevent the same class of drift that memories 96a7410 and d944d7c document in other domains.

Branch 1, Depth 2, Sub-2: Session-Scope State Propagation

Subfocus Rationale

State propagation is the invisible infrastructure of multi-agent orchestration. While file-based coordination (sibling subfocus 1) handles explicit data exchange and serialization formats (sibling subfocus 3) handle structure, this subfocus examines the implicit channels: how does a child agent come to share its parent's understanding of flags, configuration, and behavioral contracts? The parent context identified this as a rich area because the memory corpus contains several interconnected memories (ba74013, d5e503c, ba92c4e, d944d7c, b0c02ea) that together describe a layered propagation architecture unique to LLM agent systems.

Discoveries

The Three-Question Framework for Inheritance Contracts

Memory ba74013 provides the foundational framework that every session-flag spec must answer:

1. **What do subagents inherit?** — Define the subset of behaviors that propagate. Default to "all" unless scoped narrower.
2. **What breaks inheritance?** — Enumerate explicit user actions that override inherited state. Direct invocation should always be honored.
3. **Where is the contract documented?** — Place it in an `alwaysApply: true` rule so every parent and subagent loads the same contract.

This framework is deceptively simple but architecturally powerful. It forces spec authors to make inheritance explicit rather than hoping it happens implicitly.

The Push-Pull Propagation Taxonomy

Analysis of the codebase reveals two fundamentally different propagation models operating simultaneously:

Mechanism	Model	Suitable For	Failure Mode
`.crux/crux-memories.json`	Pull	Persistent thresholds, types, paths, feature flags	Hard-coding breaks consensus
`alwaysApply: true` rules	Pull (broadcast)	Behavioral contracts that span all agent types	Missing flag → silent non-propagation
Spawn-time arguments	Push	Ephemeral state, session flags, calling context	Must be enumerated in every spawn
CLI flags / overrides	Push	User-initiated overrides of defaults	Only reaches the invoked command
Phase-block ordering	Implicit	Override precedence within a loaded rule	Misordering → LLM misinterpretation

The key insight: pull propagation achieves zero-coordination consensus for persistent state, while push propagation is required for ephemeral and contextual state. The codebase uses pull for "what" (values) and push for "when/who" (session context, agent identity).

Override Precedence as Serialized Position

Memory d5e503c reveals that within a single propagated rule, override precedence is encoded as block ordering — highest-precedence first. The amnesia rule demonstrates: `Φ.amnesia (override) → Φ.enabled (default) → Φ.disabled (off)`. This is not merely a convention; it exploits the sequential nature of LLM attention: top-down reading order influences which directive is most salient.

Config as Coordination-Free Consensus

`.crux/crux-memories.json` is consumed by 48+ files across agents, skills, MCP server, hooks, evals, rules, commands, and the installer. None receive config values from a parent — each independently reads the file. This achieves consensus without coordination, with the file system as the shared data store. The pattern works because config changes are rare, consumers read at startup, and the schema is stable.

Command-Family Expansion as Propagation Maintenance

Memory 00a6d09 identifies a propagation maintenance burden: when new commands join a family, override exception lists in alwaysApply rules must be updated. The amnesia rule's exception list exists in three files (command, source rule, compressed rule). A missed update causes real behavioral suppression of the new command — not a documentation gap, but a propagation failure.

Connections

The alwaysApply ↔ Environment Variable Isomorphism

The `alwaysApply` mechanism maps precisely onto Unix process inheritance: - `alwaysApply: true` = `export VAR=value` (all children inherit automatically) - Agent file = binary-specific config (only that executable reads it) - Command file = command-line argument (only the invoked process sees it) - Non-applied rule = file on disk (accessible but never automatically read)

This isomorphism suggests that well-understood patterns from process management — explicit export, auditable inheritance chains, enumerated overrides — translate directly to agent orchestration.

The Bootstrapping Paradox (Resolved at Infrastructure Level)

A child must load the inheritance rule before it can know the rule exists. The Cursor IDE resolves this by operating at the infrastructure layer: it scans `.cursor/rules/` for `alwaysApply: true` frontmatter and injects matching rules into the system prompt before the agent processes its first token. The bootstrapping problem would resurface if rule loading were agent-driven, but the IDE pre-empts it.

A secondary bootstrap chain exists: `_CRUX-RULE.mdc` (always-applied) instructs the agent to load `CRUX.md` → agent can then interpret CRUX notation in other always-applied rules. This is a two-stage bootstrap where the IDE handles stage one and the agent handles stage two.

Broadcast Cost vs. Inheritance Completeness

Every `alwaysApply` rule consumes tokens in every agent context. The repository has ~7 always-applied rules. Adding more ensures inheritance completeness but degrades every agent's available context budget. CRUX compression partially resolves this tension: the memories integration rule achieves 55% token reduction in its `.crux.mdc` form. The compressed broadcast channel is the system's answer to the inherent cost of universal propagation.

The Knows-vs-Acts Gap as the Fundamental Challenge

The most profound discovery is that propagation mechanisms are necessary but not sufficient. Loading a rule (knowing) does not guarantee behavioral compliance (acting). Three forms of this gap were identified:

1. **Attention competition**: Multiple MUST directives in the same rule compete. The amnesia rule contains both "MUST suppress" and "MUST annotate" — the conditional logic

determining which applies depends on the LLM correctly parsing natural-language precedence.

2. **Negative assertion weakness**: Testing that a child *didn't* use memories (amnesia compliance) requires negative assertions, which are inherently ambiguous — success could mean correct suppression or irrelevant memories.
3. **Self-report unreliability**: Agents can claim compliance without achieving it (the canvas-file incident, memory 49303e0).

The only reliable detection mechanism is observability of effects: filesystem state, tracker file updates, annotation presence/absence. A compliance testing pyramid emerges from weakest to strongest: directive → self-report → output-pattern → side-effect → adversarial verification.

Child Subfocuses

Three depth-3 branches explored narrower threads:

1. **The alwaysApply Rule as an Inheritance Vehicle** (sub-4) — Why `alwaysApply: true` is the sole reliable inheritance mechanism: command files fail (invocation-scoped), agent files fail (type-scoped), non-applied rules fail (heuristic-scoped). Fragility surfaces include silent flag omission, stale exception lists, phase-block misordering, and no verification mechanism for "all contracts have alwaysApply."
2. **Config-Driven Behavior Propagation** (sub-5) — How `.crux/crux-memories.json` achieves zero-coordination consensus across 48+ consumers via pull propagation. Succeeds for persistent, slowly-changing state; fails for ephemeral, contextual, or high-frequency state. The primary failure mode is drift when consumers hard-code instead of reading config. Prevention requires testing that consumers actually pull (96a7410) and sourcing derivatives from config keys (ba92c4e).
3. **The Knows-vs-Acts Gap** (sub-6) — The fundamental disconnect between loading a directive and complying with it. Observability of effects (not intentions) is the only reliable bridge. Identified a compliance testing pyramid (directive < self-report < output-pattern < side-effect < adversarial verification) and the attention competition model where spawn-time prompts and alwaysApply rules compete for behavioral influence.

Child Insights

From sub-4 (alwaysApply as Inheritance Vehicle)

The rule-loading taxonomy is architecturally definitive: `alwaysApply` is the only mode where the IDE (not the agent) guarantees universal injection. This solves the bootstrapping problem at the infrastructure layer. The fragility surface is real but bounded — silent flag omission, exception list staleness, and phase-block misordering are all detectable by audits. The broadcast-cost tension (more rules = less context budget per agent) is partially resolved by CRUX compression but creates an implicit ceiling on how many behavioral contracts can be propagated.

From sub-5 (Config-Driven Propagation)

The push-pull spectrum is the most clarifying framework. Pull propagation (config files) achieves coordination-free consensus but only works for persistent state. Push propagation (spawn arguments, CLI flags) is required for ephemeral state but scales poorly (every spawn must enumerate). The hybrid pattern — pull for defaults, push for overrides — seen in the plugin system (b0c02ea) represents the mature architecture. The eval fixture pattern (`conftest.py` creating synthetic config at the expected path) proves the pull contract is clean: the filesystem IS the injection point.

From sub-6 (The Knows-vs-Acts Gap)

The dual-MUST conflict in the amnesia rule is a concrete instance of the gap: two MUST directives in the same loaded rule compete, and the conditional logic depends on LLM attention dynamics. The reference tracking hook provides partial observability (detecting annotation presence) but cannot detect annotation absence (agent influenced by memory but didn't annotate). The hallucination redflag (3bf625d) generalizes the gap beyond state propagation: any behavioral constraint is subject to the same knows-vs-acts failure mode.

Summary

Session-scope state propagation in this codebase operates through a layered architecture with two primary models: **pull propagation** (shared config files read independently by all agents, achieving zero-coordination consensus for persistent state) and **push propagation** (alwaysApply rules for behavioral contracts, spawn-time arguments for ephemeral state). The three-question framework (what inherits, what breaks inheritance, where is the contract) from memory ba74013 provides the design discipline, while override precedence is encoded through phase-block ordering within rules (d5e503c).

The `alwaysApply: true` mechanism is isomorphic to Unix environment variable export — the sole reliable broadcast channel because the IDE (not the agent) guarantees injection. Its fragility lies in silent degradation: a missing flag, a stale exception list, or misordered phase blocks break inheritance without error signals. Config-driven pull propagation achieves remarkable coordination-free consensus across 48+ consumers but fails for ephemeral and contextual state.

The deepest finding is the **knows-vs-acts gap**: propagation mechanisms ensure agents *load* directives, but compliance depends on LLM attention dynamics, conditional parsing accuracy, and the competition between multiple MUST directives. The only reliable bridge is observability of effects — filesystem checks, tracker state, output-pattern analysis — structured as a compliance testing pyramid from weakest (directive trust) to strongest (adversarial verification). The current system operates at levels 1-3; advancing critical propagation checks to levels 4-5 would close the gap between "the parent knows X" and "the child acts on X."

Branch 1, Depth 2, Sub-3: Serialization Formats and Data Contracts

Subfocus Rationale

The parent subfocus examines how multi-agent systems pass state between parent and child agents. Siblings cover the coordination protocols (file paths, polling, verification) and session-scope propagation (flags, config inheritance). This subfocus addresses the orthogonal concern of *how the data itself is structured* — the serialization formats, metadata contracts, provenance mechanisms, and schema evolution strategies that make inter-agent data exchange reliable regardless of which coordination protocol delivers it.

Discoveries

1. Prose-Defined Schema with Procedural Enforcement

The CRUX memory system defines its frontmatter schema entirely in prose — markdown tables in `crux-skill-memory-crud/SKILL.md` specifying 9 required fields and 4 optional fields with type constraints. There is no JSON Schema, Protobuf, or TypeScript interface. Instead, the Validate operation acts as a runtime schema checker: an LLM agent reads the prose rules and applies them. This is a deliberate design for a system where the primary consumer is an LLM, not a compiler.

Key constraints are enforced through triple-reinforcement — the same rule stated declaratively (schema table), procedurally (update steps), and proscriptively (anti-patterns). This redundancy compensates for the probabilistic nature of LLM parsing.

2. Additive-Only Schema Evolution Without Versioning

The frontmatter schema has undergone at least three waves of additive evolution: - **Wave 1 — Type transitions**: `promoted_from` added for lifecycle tracking - **Wave 2 — Consolidation**: `consolidated_from`, `consolidation_topic`, `consolidation_part` added as a cohesive sub-schema - **Wave 3 — Compression**: 7 fields (`compressed`, `compressionTarget`, `beforeTokens`, `afterTokens`, `reducedBy`, `compressedDate`, `sourceArchive`) added by a different skill entirely

Each wave added optional fields. Existing consumers that don't know about new fields simply skip them — the YAML equivalent of Protobuf's unknown field preservation, achieved without version numbers, schema migration, or generated code. The required/optional partition maps to the Open-Closed Principle: closed for modification (required fields never change), open for extension.

3. Immutability as a Cross-File Coordination Primitive

Two fields are designated write-once: `id` (sha256(title)[:7]) and `created`. The `id` survives title edits, making it a stable foreign key that reference trackers, consolidation records, and the memory index all rely on. This serves the same function as immutable event IDs in event-sourced systems — stable reference points that multiple independent agents can use without coordination.

4. Six-Layer Provenance Model

The system implements comprehensive provenance through explicit fields rather than path/timestamp inference:

Layer	Mechanism	Question Answered
Origin	`source` field ("spec-slug" or "adhoc")	Who created this?
Merge	`consolidated_from` (list of original IDs)	What was it made of?
Transition	`promoted_from` / `demoted_from`	What did it used to be?
Compression	`sourceArchive`, `sourceChecksum`	Where is the original?
Usage	`.refs.yml` with polymorphic source keys	Who consumed it?
Generation	`generated` timestamp + banner	When was this derived?

These form a directed acyclic provenance graph that survives every mutation: file moves, compression, consolidation, promotion. Explicit provenance was deliberately chosen over inference — paths change during type transitions, timestamps collide across timezones, and content similarity is expensive to compute.

5. Three-Format Architecture Optimized for LLMs

The system uses a deliberate format hierarchy matching different consumers:

```
CRUX notation   → token-constrained LLM (5-10x compression)
Markdown body   → LLM + human (zero deserialization needed)
YAML frontmatter → human diff + machine indexing (readable, scannable)
JSON config      → CI/CD, install scripts (strict, no ambiguity)
```

The choice of markdown+YAML over pure JSON or protobuf reflects the primary consumer: LLMs interpret markdown tables and procedural instructions more reliably than formal schema definitions. The cost is real — no IDE autocomplete, no compile-time guarantees, YAML type coercion risks (memory 96a7410 documented drift between tooling defaults and spec) — but bounded by the fact that the primary "parser" is robust to ambiguities that would break traditional parsers.

6. Eventual Consistency Over Strict Enforcement

The system consistently chooses repair over prevention for data integrity: - Orphaned trackers → cleaned up during REM sleep, not prevented by transactions - Type placement mismatches → detected by Validate, not prevented by directory constraints - Stale `consolidated_from` → points to archived (not deleted) memories - Cross-file references → convention-based (slug matching), not foreign key enforced

This mirrors memory d944d7c's lesson: drift is inevitable in multi-agent environments, so build repair mechanisms rather than prevention mechanisms.

Connections

Data Contracts as Agent Communication Protocol

The frontmatter schema functions as an implicit communication protocol between agents. The CRUD skill is the "writer API," the index builder is the "reader API," and the Validate operation is the "protocol checker." No network protocol or message queue is needed — the shared file system with agreed-upon field contracts provides equivalent guarantees for an asynchronous, crash-prone multi-agent system.

Config-Driven Enums Prevent a Class of Drift

Memory ba92c4e captures a key anti-drift pattern: when interactive options must mirror a domain concept (like memory types), source the options from config keys. Adding a new memory type to `typeTransitions` in config automatically propagates to the UI, Validate, and directory structure. This is schema evolution at the enum constraint level — the set of valid values for a typed field grows without touching the contract definition. Combined with memory 96a7410's warning about tooling-spec drift, the pattern that emerges is: **single-source-of-truth for constraint definitions, procedural enforcement at every consumption point**.

The Provenance Graph Enables Intelligent Downstream Decisions

Provenance isn't just audit — it drives operational logic: - REM sleep applies longer dormancy to `source: "adhoc"` memories (they lack peer review) - Memories with diverse reference sources resist demotion - Consolidated memories preserve full ancestry for conflict resolution - `sourceArchive` enables exact restoration of compressed memories

CRUX Notation as a Novel Fourth Format Layer

CRUX creates a format that is neither human-readable (requires the CRUX.md key) nor machine-parseable by conventional tools (no parser exists; the "parser" is an LLM). It is purpose-built for a consumer class that didn't exist when traditional serialization formats were designed: a token-counting LLM that reads sequentially and benefits from semantic density. Memory d5e503c shows that even ordering within CRUX blocks carries semantic weight — a property impossible in key-value formats where ordering is undefined.

Git Trackability as an Architectural Constraint

All format layers share one hard constraint: they must be plain text that git can diff, merge, and track. This eliminates binary formats, database-backed stores, and deeply nested structures. The system trades parsing strictness for collaboration tooling compatibility.

Memory 6c16dc6 (adversarial verification catches gaps at 25% hit rate) depends on this — verification runs on diffable text files.

Child Subfocuses

Three narrower threads were explored at depth 3:

Sub-7: YAML Frontmatter as a Metadata Contract

How the 9-required / N-optional field partitioning creates forward-compatible contracts without schema versioning. Examined immutability rules, additive evolution waves, eventual consistency for referential integrity, and the triple-reinforcement pattern for LLM-consumed constraints.

Sub-8: Provenance Tagging and Multi-Writer Disambiguation

How `source`, `consolidated_from`, `promoted_from`, `sourceArchive`, and `.refs.yml` attribution create a six-layer provenance DAG where every transformation is recorded explicitly. Examined the design principle of explicit over implicit provenance and its survival across file moves, compression, and consolidation.

Sub-9: The Tension Between Human-Readable and Machine-Parseable Formats

Why the system uses markdown+YAML over pure JSON or protobuf, and the three-format stack (JSON config, YAML frontmatter, markdown body) plus CRUX as a fourth layer. Examined the consumer-optimized format pyramid, the cost of prose-defined schemas, and git trackability as an architectural constraint.

Child Insights

From Sub-7 (YAML Frontmatter as Contract)

The prose-defined schema works because the primary "parser" is probabilistic — an LLM benefits from the same constraint stated in declarative, procedural, and proscriptive framings simultaneously. The required/optional partition is the key to forward compatibility: three waves of schema evolution (promotion, consolidation, compression) were each absorbed without breaking any existing consumer. Immutability of `id` and `created` provides referential stability analogous to immutable event IDs in event-sourced systems. Cross-file references use eventual consistency (REM sleep cleanup) rather than strict foreign key constraints — a pragmatic choice for a crash-prone multi-agent environment where an agent might fail between creating a memory and its tracker.

From Sub-8 (Provenance Tagging)

Six distinct provenance mechanisms collectively ensure no memory's lineage depends on inference from paths, timestamps, or content. The provenance fields form a DAG: `source → consolidated_from → promoted_from → sourceArchive → .refs.yml`. Each layer answers a different lifecycle question. The core design principle is that explicit provenance survives every mutation the system performs (moves, compression, consolidation, promotion), while inference-based provenance degrades at each step. The `source` field pattern generalizes directly to any shared store with multiple writers. Reference trackers generalize similarly — recording both who wrote and who read creates a bidirectional provenance graph without centralized coordination.

From Sub-9 (Human vs Machine Format Tension)

The format choices reflect an optimization hierarchy: LLM comprehension > human reviewability > git trackability > machine parsing strictness. JSON handles the narrow case where ambiguity is unacceptable (config). YAML frontmatter handles metadata that must be both indexable and diffable. Markdown handles the primary consumer — LLMs interpreting definitions without deserialization. CRUX handles the edge case where even markdown is too token-expensive. The "no formal schema" choice is deliberate: adding `consolidated_from` required updating only prose documentation; the LLM understood immediately without schema migration. The cost (type coercion, drift, no autocomplete) is bounded by the LLM's robustness to ambiguities that would break traditional parsers.

Summary

Serialization formats and data contracts in this multi-agent system are built on four key architectural decisions: (1) **prose-defined schemas with procedural enforcement** — frontmatter contracts are documented in markdown and enforced by LLM-agents running Validate operations, optimizing for the primary consumer (LLMs) over traditional parsers; (2) **additive-only schema evolution** — a required/optional field partition enables three waves of capability addition without version numbers, migration scripts, or breaking changes; (3) **explicit six-layer provenance** — `source`, `consolidated_from`, `promoted_from`, `sourceArchive`, `.refs.yml` attribution, and `generated` banners create a complete lifecycle DAG where no lineage question requires inference from paths or timestamps; and (4) **a consumer-optimized format stack** — JSON for strict machine config, YAML frontmatter for diffable metadata, markdown for LLM-interpretable content, and CRUX notation for token-constrained contexts. The system consistently trades parsing strictness for LLM comprehension, repair (REM sleep) for prevention (foreign keys), and prose documentation for formal schemas — choices that reflect a world where the most important data consumer is not a compiler but a language model.

Branch 1, Depth 3, Sub-1: Polling-for-Existence vs Event-Driven Notification

This codebase deploys three detection patterns across a latency spectrum — watchdog events (~1s) for live MCP queries, ls-polling (10-30s) for agent-to-agent meditation coordination, and pending-flag files (session-granularity) for cross-session state. Each is well-matched to its use case's latency tolerance.

The key insight is that **detection alone is insufficient**. Memory 49303e0 (silent Write tool failures) reveals that file existence doesn't guarantee file completeness. Neither pattern includes a content validation layer. The watchdog's 1-second debounce accidentally mitigates write-completeness races (by waiting past the CREATE→CLOSE_WRITE event sequence), but polling has no equivalent safeguard.

The most actionable gap is in meditate's polling loop: there is no timeout, no abort-on-missing-child, and no file-integrity check after detection. A child agent crash silently deadlocks the parent. Adding `max_wait_time` with graceful degradation (proceed with N-1 branches) would make the coordination protocol robust against the failure modes this codebase has already documented.

Branch 1, Depth 3, Sub-2: Write Verification and Persistence Guarantees

Write verification is the weakest link in file-based agent coordination. A real incident (memory 49303e0) proved that the Write tool can silently fail, making post-write verification mandatory.

The codebase contains the building blocks of a reliable write pipeline — pre-write archival (cd0c954), staging directories (install.py), checksum verification (crux-utils + install.py), and a three-layer verification protocol (ls → Read → git status) — but these exist as scattered patterns rather than a unified primitive. The key architectural insight is that archive-before-write + verify-after-write creates a transactional envelope that converts an unreliable write operation into a recoverable one. Idempotency enables safe retries, but side-effectful operations like archival need their own idempotency guards. The gap is the absence of a formal `write-and-verify` skill that all agents invoke consistently, rather than each skill reimplementing verification ad-hoc.

Branch 1, Depth 3, Sub-3: Path Convention Design and the Coordination Namespace

Path conventions are the addressing layer of file-based agent coordination — they make decoupled coordination possible by enabling consumers to independently compute expected file locations. This codebase uses five distinct convention families, each optimised for its domain: positional encoding for tree structures, slug-based naming for content identity, config-sourced templates for user-customisable patterns, extension-based type discrimination for variant encoding, and sentinel files for cross-session state signals. Good conventions share four properties: determinism (same inputs → same path), collision avoidance (no two writers target the same path), human readability (tree position visible at a glance), and machine parseability (regex-extractable structured data). The primary risk is convention drift — when a path template is defined in one place and reconstructed in another, silent divergence can cause agents to write and read at different locations. The codebase already exhibits one live instance of this: the meditation depth-3 file tree diagram implies globally sequential sub-indices while the protocol description implies per-parent-relative indices. Centralising path templates (as the dream system does via `summaryPattern` in config) mitigates drift; hardcoding in prose (as the meditation system does) invites it.

Branch 1, Depth 3, Sub-4: The alwaysApply Rule as an Inheritance Vehicle

`alwaysApply: true` is the sole reliable inheritance vehicle for behavioral contracts across arbitrary subagent types because it is the only rule-loading mode where the IDE — not the agent — guarantees universal injection. Command files fail (invocation-scoped), agent files fail (type-scoped), and non-applied rules fail (heuristic-scoped). The mechanism solves the bootstrapping problem by operating at the infrastructure layer, outside agent control. Its fragility lies in silent degradation: a missing flag, a stale exception list, or a misordered phase block all break inheritance without any error signal. The compressed broadcast channel (CRUX `.crux.mdc`) partially resolves the context-budget tension that broadcast universality creates. The pattern is isomorphic to Unix environment variable inheritance, suggesting that

established process-management principles (explicit export, auditable inheritance, enumerated overrides) are directly applicable to agent orchestration.

Branch 1, Depth 3, Sub-5: Config-Driven Behavior Propagation

Config-driven behavior propagation (pull model) achieves zero-coordination consensus across 48+ independent consumers in this codebase. Its power comes from treating the file system as a shared data store where all agents independently read the same truth. The pattern succeeds for persistent, slowly-changing, schema-stable state (feature flags, thresholds, type definitions, paths). It fails for ephemeral, contextual, or high-frequency state — which is why the codebase uses push propagation (spawn arguments, alwaysApply rules, CLI overrides) for session flags and calling context. The primary failure mode is drift: when a consumer hard-codes rather than reads, the consensus breaks silently. Prevention requires testing that consumers actually pull (memory 96a7410) and sourcing all derivative artifacts from config keys (memory ba92c4e). The hybrid pattern — pull for defaults, push for overrides — seen in the plugin system (memory b0c02ea) represents the mature architecture: config provides the baseline, explicit signals override it.

Branch 1, Depth 3, Sub-6: The Knows-vs-Acts Gap

The knows-vs-acts gap is the fundamental challenge in agent state propagation: having a directive loaded into context does not guarantee behavioral compliance. This repository's own evidence demonstrates the gap in multiple forms — file creation claims without filesystem effects (49303e0), tooling defaults that drift from specifications (96a7410), and dual-MUST conflicts where conditional overrides compete with base directives in the same rule.

The only reliable detector is observability of effects, not of intentions. The reference tracking hook provides partial observability for memory usage, but negative compliance (suppression during amnesia) is inherently harder to verify than positive compliance. The eval suite tests amnesia inheritance (P3) using output absence checks, but these are weakened by ambiguous negative assertions.

A compliance testing pyramid emerges: directive < self-report < output-pattern < side-effect < adversarial verification. The current system operates mostly at levels 1-3; pushing more critical propagation checks to levels 4-5 would increase confidence that inherited state actually influences child behavior. The key design insight is that the highest-propagation-guarantee channel (alwaysApply rules) and the highest-salience channel (spawn-time prompt) may not be the same, and both are needed for reliable compliance.

Branch 1, Depth 3, Sub-7: YAML Frontmatter as a Metadata Contract

YAML frontmatter works as an inter-agent metadata contract because it implements five key design choices: (1) a fixed set of 9 required fields forming a minimum viable contract that

every consumer can depend on; (2) an open-ended set of optional fields that enable additive-only schema evolution without version numbers or migration; (3) write-once immutability on `id` and `created` providing stable cross-file reference anchors; (4) prose-defined schema with procedural enforcement via the Validate operation, optimised for LLM consumption over formal machine parsing; and (5) eventual consistency for referential integrity, preferring periodic repair (REM sleep) over strict write-time constraints. The schema has already undergone three waves of additive evolution (promotion tracking, consolidation metadata, compression metadata) without breaking any existing consumers — the same forward-compatibility property that JSON Schema and Protobuf achieve through formal versioning, achieved here through the simpler mechanism of optional YAML keys that unknown consumers silently skip.

Branch 1, Depth 3, Sub-8: Provenance Tagging and Multi-Writer Disambiguation

The CRUX memory system implements a comprehensive, layered provenance model where every transformation — creation, consolidation, type transition, compression, consumption — is recorded as an explicit frontmatter field or tracker entry. Six distinct provenance mechanisms (`source`, `consolidated_from`, `promoted_from`, `sourceArchive`/`sourceChecksum`, `.refs.yml` attribution, `generated` banner) collectively ensure that no memory's lineage depends on inference from paths, timestamps, or content analysis. The core design principle is that explicit provenance survives every mutation the system can perform — file moves, compression, consolidation, promotion — while inference-based provenance degrades at each step. This pattern generalizes to any multi-agent architecture where shared data stores receive writes from multiple sources: tag origin at write time, record attribution at read time, and preserve transformation history at every intermediate step.

Branch 1, Depth 3, Sub-9: Human-Readable vs Machine-Parseable Format Tension

The system's format choices reflect a deliberate optimization hierarchy: **LLM comprehension > human reviewability > git trackability > machine parsing strictness**. JSON handles the narrow case where machine parsing must be unambiguous (config). YAML frontmatter handles the case where structured metadata must be both machine-indexable and human-diffable. Markdown bodies handle the primary consumer — LLMs interpreting agent/skill/memory definitions without deserialization. CRUX notation handles the edge case where even markdown is too token-expensive. The cost is real (no formal schemas, type coercion risks, drift between prose specs and implementations) but bounded by the fact that the primary "parser" — the LLM — is robust to these ambiguities in ways traditional parsers are not. The format stack is optimized for a world where the most important code reader is not a compiler but a language model.

---

4. Branch 2: Failure Handling and Resilience

Branch 2 — Depth 1 Synthesis

Subfocus Rationale

Failure handling is the critical dimension that separates toy agent orchestration from production-grade harnesses. While state coordination (sibling 1) addresses the happy path and resource governance (sibling 3) addresses prevention, this branch explores what happens when things go wrong despite those measures — how harnesses detect failures, distinguish transient from permanent, decide between automatic recovery and escalation, and maintain system integrity under partial failure conditions.

Discoveries

The Four-Layer Resilience Stack

[memory:Exponential backoff with jitter on rate-limit errors] [memory:Per-phase parallel subagent execution reduces wall-clock time] [memory:Gate expensive SDK evals behind SDK_EVAL_SKIP_EXPENSIVE]

The CRUX eval harness implements a mature four-layer defence-in-depth architecture:

Layer	Mechanism	Examples	Role
Demand reduction	Shared-agent-runs, skip-by-default gating	N calls → 1; expensive tests opt-in only	Eliminate unnecessary API load
Concurrency limiting	maxForks: 2, dependency-based phasing	Phase-parallel execution, conservative fork counts	Reduce peak simultaneous pressure
Reactive recovery	Exponential backoff + jitter on rate-limit detection	`withRetry()`: BASE=2s, MAX=60s, 5 retries, ±15% jitter	Recover from limits hit despite proactive measures
Hard backstop	Global wall-clock deadline	`SDK_EVAL_MAX_DURATION_MS`: 60min → `process.exit(1)`	Kill suite if reactive layer loops too long

The Trust Inversion Principle

[memory:Agent-reported file creation must be verified on disk] The deepest epistemological finding: **an agent's narrative is a hypothesis, not evidence.** An executing agent reported a file as created with full contents and path, yet the file never existed on disk. This demands a fundamental trust inversion — treat all agent claims as falsifiable and verify against ground truth (filesystem state, not self-report).

The 25% Hit Rate Signal

[memory:Adversarial verification catches real documentation gaps] Independent verification by a separate agent found 6 issues in 24 subtasks (25% overall), but with high variance: 7% in isolated implementations, 50% in cross-cutting integrations. The failure rate correlates with integration complexity, not task count.

Semantic Hang Detection Gap

The existing harness has one semantic detector (AskQuestion abort) but four unaddressed patterns: tool-call loops, thinking-without-acting, file-polling deadlocks, and phantom completions. All share a common trait: time-based defences alone cannot distinguish them from legitimate slow work.

The All-or-Nothing Problem

Current architecture has no intermediate state between "agent is struggling" and "kill everything": - Meditation polls forever for child output files — zero timeout - Spec execution phases block until all agents complete - Shared test runs cascade-fail all N dependent tests on beforeAll failure - `withRetry` only handles rate limits — all other errors throw immediately

Connections

Reactive Retries as Feedback Signal (not just recovery)

The "reduce forks rather than increase retries" guideline from memory `6265f8f` reveals that retry frequency functions as a **tuning signal** for proactive measures. If retries fire frequently, the proactive settings (fork count, batching) need adjustment. Removing retries would remove this feedback loop. This generalises to "reduce demand rather than increase tolerance" — a principle that applies beyond test harnesses to any agent orchestration system hitting resource limits.

Cross-Reference Divergence as Structural Failure Pattern

All five categories where adversarial verification excels (silent persistence, documentation drift, cross-file consistency, default/spec alignment, distribution completeness) share a structural property: **truth distributed across multiple files** that a single sequential agent cannot hold in working memory simultaneously. The verifier's advantage is fresh context with no prior commitment to either source being correct.

Progress Tokens as Unifying Hang Detection

All semantic hang patterns reduce to one question: "Is the agent making forward progress?" A `lastProgressTimestamp` tracker based on novel tool calls, new files written, or novel output text would subsume tool-call loops, thinking-without-acting, and file-polling deadlocks into a single configurable mechanism — far more robust than pattern-matching individual failure modes.

The Exploratory/Transactional Divide

The decision to proceed with partial results depends on operation nature: - **Exploratory** (meditation, recall, research): Partial results almost always beat nothing. Annotate the gap, proceed. - **Transactional** (spec execution, memory creation): Partial results risk inconsistent state. Escalate to user.

This maps directly onto the meditation protocol's file-based coordination advantage: child outputs persist independently, enabling selective retry at the lowest possible level without reconstructing state.

Concurrency as Propagating Constraint

Concurrency management propagates through the entire stack: test design (shared runs) → test selection (skip gates) → runtime config (maxForks) → retry parameters (jitter width) → hard limits (wall-clock deadline). All must be coherent — narrow ±15% jitter is adequate for

maxForks: 2 but fails catastrophically at maxForks: 4 (56% collision probability). Session flags and concurrency limits need explicit inheritance contracts so child agents don't accidentally violate parent constraints.

Child Subfocuses

1. Retry Strategies and Transient Error Recovery

Rationale: Retry logic is the most concrete, implementable dimension — it has a real implementation in the harness with quantifiable parameter tradeoffs and a clear path from current state to improvements.

2. Timeout Detection, Hang Recovery, and Graceful Degradation

Rationale: The gap between the existing three-tier timeout hierarchy and what's needed for robust multi-agent orchestration is the most urgent architectural concern — a single hung leaf can block an entire meditation tree indefinitely.

3. Adversarial Verification as Post-Hoc Failure Detection

Rationale: With 25% empirical hit rate and documented cases of catching completely invisible failures, adversarial verification is the most evidence-backed failure detection mechanism in the corpus and warrants dedicated cost-benefit analysis.

Child Insights

From Sub-1: Retry Strategies and Transient Error Recovery

The harness's actual retry budget is 62s nominal (not the documented 122s — off-by-one in the memory), consuming only ~21% of a 300s test timeout. Three concrete improvements emerged:

1. **Widen jitter** from ±15% to ±50% or decorrelated jitter to safely support higher parallelism
2. **Raise base delay** from 2s to 5s to reduce wasted retries against typical 10–60s rate-limit windows
3. **Evolve error classifier** from binary (rate-limit or throw) to three-tier (definitely transient → full budget, probably transient → short budget, definitely permanent → throw)

The deeper principle: reactive retries and proactive demand reduction form a feedback loop — retry frequency signals whether proactive settings are correct. Removing retries removes the diagnostic signal.

The four-layer stack (demand reduction → concurrency limiting → reactive retry → hard deadline) composes multiplicatively: 6x demand reduction × 2-fork limit = 12x reduction in rate-limit probability. Demand reduction has compounding returns (fewer calls = lower cost AND fewer failures), while concurrency reduction has diminishing returns (serial execution penalises wall-clock).

From Sub-2: Timeout Detection, Hang Recovery, and Graceful Degradation

The harness has solid time-based defence (3-tier timeout hierarchy) but critical gaps:

Five semantic hang patterns were catalogued with detection mechanisms: 1. Tool-call loops (hash name+args; 3 identical = warning, 5 = abort) 2. Output stagnation (cosine similarity >0.9 over 3+ output cycles) 3. Thinking-without-acting (10+ consecutive thinking events without tool calls after 60s) 4. File-polling deadlocks (same path checked >10 times without success) 5. Phantom completions (Write tool "completed" but file doesn't exist)

A **unifying progress token framework** — track `lastProgressTimestamp` based on novel actions — subsumes all five into a single configurable mechanism.

The most urgent fix: Add a polling timeout to the meditate protocol's file-based coordination so a single hung leaf cannot block the entire meditation tree indefinitely.

Partial-result decision matrix: failure_mode × branch_criticality × correlation → action. Correlated failures (all N branches) = systemic → escalate immediately. Isolated failure + non-critical = degrade gracefully. The graceful consolidation pattern — proceed with available results, annotate the gap — is strictly better than all-or-nothing for exploratory operations.

From Sub-3: Adversarial Verification as Post-Hoc Failure Detection

Adversarial verification excels at detecting **cross-reference divergence** — failures where truth is distributed across multiple files. Five categories are well-served; semantic correctness, performance, and security are not.

Graduated verification model optimizes cost-benefit: - Tier 1 (near-zero cost): Automated filesystem checks on ALL subtasks (ls, git status, size checks) - Tier 2 (moderate cost): Targeted adversarial review on HIGH-RISK subtasks (risk score ≥ 3) - Tier 3 (expensive): Deep specialized audit on CRITICAL subtasks (risk score ≥ 5)

Risk heuristic: File creation (+3), cross-cutting registration (+3), documentation with paths (+2), first-time agent (+2), spec misalignment history (+2).

Recovery vs escalation boundary decomposes along three axes: - Mechanistic failures (file not persisted) → auto-recover - Judgmental failures (spec contradicts subtask) → escalate - Expensive retries → always escalate regardless of confidence

Advisory auto-recovery pattern: fix the problem AND log it prominently, with a circuit breaker (per-category retry counter) that forces escalation on recurrence. This preserves both velocity and diagnostic visibility.

Known exceptions manifest (analogous to `.eslintignore`) suppresses false positives from pre-existing baseline noise, preventing verifier fatigue.

Summary

Failure handling in multi-agent workflows operates through a defence-in-depth stack with four complementary layers: demand reduction, concurrency limiting, reactive recovery, and hard backstops. The CRUX harness implements all four but has three critical gaps:

1. **Semantic hang detection**: Beyond the single AskQuestion abort pattern, five more hang patterns are undetected. A "progress token" framework would unify detection into a single configurable mechanism. The most urgent concrete fix is adding a polling timeout to file-based coordination.
2. **Partial-result degradation**: The current all-or-nothing model wastes successful work. For exploratory operations, graceful consolidation (proceed with available results, annotate gaps) is strictly better. For transactional operations, correlation-first triage (all-fail = systemic → escalate, one-fail = isolated → retry leaf) prevents the most wasteful failure pattern.
3. **Systematic verification**: Adversarial verification's 25% hit rate justifies its cost, but uniform application is wasteful. A graduated model (automated checks on all, targeted adversarial on high-risk, deep audits on critical) with a risk scoring heuristic optimizes the cost-benefit curve.

The deepest cross-cutting insight: **an agent's narrative is a hypothesis, the filesystem is ground truth, and retry frequency is a diagnostic signal**. These three principles — trust inversion, ground-truth designation, and feedback loops — form the epistemological foundation for any resilient agent orchestration system.

Branch 2, Depth 2, Sub-1: Retry Strategies and Transient Error Recovery

Subfocus Rationale

Retry logic is the most concrete, implementable dimension of failure resilience — it operates at the boundary between "detect" and "recover" and its parameters have quantifiable tradeoffs. Unlike timeout detection or graceful degradation (siblings in the parent facet), retry strategies have a real implementation in the CRUX harness (`evals/sdk/helpers/harness.ts`) with documented design decisions, making them ideal for grounded analysis rather than abstract pattern-cataloguing.

Discoveries

The CRUX harness as a concrete case study

[memory:Exponential backoff with jitter on rate-limit errors] The harness at `evals/sdk/helpers/harness.ts` exports `withRetry()` and `sendWithRetry()` — a clean separation between the generic retry wrapper and the agent-specific entry point. The implementation makes three core design choices:

1. **Error classification by message inspection**: `isRateLimitError()` checks `err.message.toLowerCase()` against five substring patterns (rate limit, rate_limit, 429, too many requests, throttl). Non-matching errors are thrown immediately — no retry.
2. **Exponential backoff with bounded jitter**: `BASE_DELAY_MS=2000`, exponential doubling, `MAX_DELAY_MS=60000` cap, ±15% jitter spread.
3. **Fixed retry budget**: `DEFAULT_MAX_RETRIES=5` — after 5 failed retries, the error propagates.

Key quantitative finding: the budget is more conservative than documented

Memory `6265f8f` states worst-case delay is ~122s. The depth-3 analysis corrected this to **62s nominal (53–71s with jitter)** — the code throws at attempt 5 before computing a 6th delay. This means the retry budget consumes only ~21% of a 300s test timeout, leaving substantial headroom. The `MAX_DELAY_MS=60000` cap is effectively dead code under current parameters (never reached with only 5 retries).

Three-tier classification gap

The current classifier is binary: rate-limit errors retry, everything else throws. Server errors (500, 502, 503), network errors (ECONNRESET, ETIMEDOUT), and DNS failures are all classified as permanent. For a test harness this is defensible (fail fast, investigate). For a production agent harness, a three-tier model would serve better: (1) definitely transient → full retry budget, (2) probably transient → short retry budget (1–2 attempts), (3) definitely permanent → throw immediately.

Jitter width couples to parallelism

The ±15% jitter spread produces a 30% delay window at each attempt. At `maxForks: 2` the collision probability is ~10% — acceptable. At `maxForks: 4` it jumps to 56%, making thundering-herd retries likely. The `maxForks: 2` recommendation and the ±15% jitter form a tightly coupled pair — changing either requires re-evaluating the other. Wider alternatives (±50%, full jitter, decorrelated jitter) would decouple this constraint.

Four-layer resilience stack

The harness implements not one but four complementary resilience layers:

Layer	Mechanism	Role
Demand reduction	Shared-agent-runs (N calls → 1), skip-by-default gating	Eliminate unnecessary API calls
Concurrency limiting	`maxForks: 2`, dependency-based phasing	Reduce peak simultaneous load
Reactive recovery	Exponential backoff + jitter on rate-limit detection	Recover from limits hit despite proactive measures
Hard backstop	Global `SDK_EVAL_MAX_DURATION_MS` (60min)	Kill suite if reactive layer loops too long

Connections

Retries as feedback signal, not just recovery

The "reduce forks rather than increase retries" guideline from `6265f8f` reveals that retry frequency functions as a **tuning signal** for proactive measures. If retries fire frequently, the proactive settings (fork count, batching) need adjustment. If retries never fire, the proactive settings are correct. Removing retries would remove this feedback loop — making it impossible to know whether proactive measures are sufficient.

Demand reduction vs concurrency reduction — multiplicative composition

Shared-agent-runs are demand reduction (eliminate calls entirely), while `maxForks` is concurrency reduction (same calls, fewer simultaneous). These compose multiplicatively: 6x demand reduction × 2-fork limit = 12x reduction in rate-limit probability vs unbatched 4-fork execution. This distinction matters because demand reduction has compounding returns (fewer calls = lower cost AND fewer failures), while concurrency reduction has diminishing returns (serial execution penalises wall-clock time).

The "429" false-positive and `instanceof Error` cross-realm fragility

The substring `"429"` can false-positive on non-rate-limit messages containing that number (trace IDs, line numbers). The `instanceof Error` guard fails across serialisation boundaries (child processes, JSON round-trips, worker threads). Neither is an active bug, but both are latent fragilities that would surface if the SDK's error transport evolves. A word-boundary regex (`/\b429\b/`) and a duck-typing check (`typeof err === 'object' && 'message' in err`) would harden both.

Structured error types as the ideal evolution

The sample codebase demonstrates `IntegrationError` with an explicit `retryable: boolean` property — pushing classification to the error producer who knows the failure's nature. This eliminates message inspection entirely. The current substring approach is a pragmatic workaround for consuming opaque third-party errors; the long-term direction is structured error types when the SDK provides them.

Bimodal retry importance

Skip-by-default gating creates two runtime profiles: default mode (cheap tests, retries are insurance) and expensive mode (recursive multi-agent tests, retries are load-bearing). The

retry mechanism's importance is inversely proportional to proactive gating effectiveness — exactly when you opt into expensive work, the safety net matters most.

Base delay alignment with rate-limit windows

The 2s base delay likely wastes the first 1–2 retry attempts against typical 10–60s rate-limit reset windows (cumulative delay after 2 retries is only ~14s). A base of 5s with 4 retries would maintain the same total budget (~75s) while reducing wasted attempts. The optimal base delay is `ceil(reset_window / 2)` — long enough that first retry might clear the window, short enough not to dominate the budget.

Concurrency as a propagating constraint

Concurrency management is not a single knob but a constraint that propagates through multiple system layers: test design (shared runs), test selection (skip gates), runtime config (maxForks), retry parameters (budget, jitter), and hard limits (wall-clock deadline). All must be coherent — a generous maxForks undermines demand reduction, narrow jitter fails at high parallelism, and aggressive skip-gating makes retries vestigial in default mode but essential in explicit-run mode.

Child Subfocuses

Sub-1: Error Classification Heuristics for Transient vs Permanent Failures

Rationale: Error classification is the decision gate that determines all downstream retry behaviour. Getting the gate wrong (false positive or false negative) has outsized consequences, warranting dedicated analysis of the substring-matching approach, cross-realm fragilities, and the missing three-tier classification model.

Sub-2: Backoff Parameter Tuning and Retry Budget Optimisation

Rationale: The concrete parameters (BASE_DELAY, MAX_DELAY, retries, jitter) have quantifiable tradeoffs against test timeouts, rate-limit window alignment, and fork-count collision probabilities. Mathematical analysis reveals the budget is more conservative than documented and identifies specific parameter improvements.

Sub-3: Pairing Reactive Retries with Proactive Concurrency Reduction

Rationale: Reactive and proactive strategies are often designed independently but their interaction determines whether the resilience model is synergistic or counterproductive. Understanding the four-layer stack and the feedback loop between retry frequency and proactive tuning is essential for generalising beyond test harnesses.

Child Insights

From Sub-1 (Error Classification)

- The `isRateLimitError()` binary classifier has three latent fragilities: bare `"429"` substring can false-positive, `instanceof Error` fails across serialisation boundaries, and the binary model discards "probably transient" server/network errors.
- The extensibility model (hardcoded substring list) is appropriate for <10 patterns but should evolve to a configurable array or structured error types as the error surface grows.
- The global timeout (`SDK_EVAL_MAX_DURATION_MS`) forms a defence-in-depth pair with the classifier — catching false positives that would otherwise retry indefinitely.
- The ideal long-term direction is pushing classification to the error producer via `retryable: boolean` properties on structured error types.

From Sub-2 (Backoff Parameter Tuning)

- The actual worst-case delay budget is **62s nominal**, not the 122s documented in the memory — the code throws at attempt 5 before computing a 6th delay. This off-by-one means the parameters are more conservative than intended.
- The `MAX_DELAY_MS=60000` cap is never reached under current parameters (max un-jittered delay at attempt 4 is 32s).
- The ±15% jitter is adequate for `maxForks: 2` (10% collision probability) but fails at 4+ forks (56%). Widening to ±50% or switching to decorrelated jitter would allow safe parallelism increase.
- A base delay of 5s with 4 retries would maintain the same total budget while reducing wasted attempts against 10–60s rate-limit windows.
- The shared-agent-runs pattern is a structural prerequisite for the narrow jitter — without it, effective concurrency would exceed what ±15% jitter can safely desynchronise.

From Sub-3 (Reactive + Proactive Pairing)

- The four-layer stack (demand reduction → concurrency limiting → reactive retry → hard deadline) operates as defence-in-depth where each layer catches what the previous misses.
- Retries are never vestigial — they function as both a recovery mechanism and a feedback signal for tuning proactive measures. Removing retries breaks the feedback loop.
- The "reduce forks rather than increase retries" guideline generalises to "reduce demand rather than increase tolerance" — a principle that applies to production API clients, multi-agent orchestration, and connection pool management.
- Skip-by-default creates a bimodal profile where retry importance is inversely proportional to proactive gating effectiveness.

- Concurrency limits, like session flags, need explicit inheritance contracts in multi-agent systems — a parent limiting to maxForks: 2 is undermined if child agents each spawn their own parallel work.

Summary

The CRUX harness implements a mature four-layer resilience stack: demand reduction (shared runs, skip gating) → concurrency limiting (maxForks: 2, phased execution) → reactive retry (exponential backoff + jitter on rate-limit detection) → hard backstop (global wall-clock deadline). The actual retry budget is 62s nominal, not the documented 122s, leaving 75%+ of test timeout for real operations. Three improvement opportunities emerged: (1) widen jitter from ±15% to ±50% or decorrelated to enable safe fork-count increase, (2) raise base delay from 2s to 5s to reduce wasted retries against typical rate-limit windows, (3) evolve the binary error classifier toward a three-tier model (definitely transient / probably transient / definitely permanent) for broader error coverage. The deepest insight is that reactive retries and proactive measures form a feedback loop — retry frequency is the signal that tells you whether your proactive settings are correct — making retries essential even when proactive measures work perfectly. The "reduce demand rather than increase tolerance" principle generalises from test harnesses to any agent orchestration system hitting resource limits.

Branch 2, Depth 2, Sub-2: Timeout Detection, Hang Recovery, and Graceful Degradation

Subfocus Rationale

Timeout and hang detection are the active defence layer in agent orchestration — they determine how quickly a harness recognises that something has gone wrong, and what it does next. This narrowing was chosen because the current CRUX harness has a rich but incomplete timeout hierarchy (per-test, per-hook, global deadline, plus one semantic detector) with critical gaps: no polling timeouts, no partial-result degradation, and no detection of phantom completions. Closing these gaps is prerequisite to reliable multi-agent workflows.

Discoveries

The Three-Tier Timeout Hierarchy (Existing)

The CRUX eval harness implements three time-based defence layers:

Tier	Mechanism	Default	Purpose
Per-test	vitest `testTimeout` / per-test `{ timeout: N }`	240s (up to 480s for meditate)	Kill individual hung agents
Per-hook	vitest `hookTimeout`	120s	Kill stuck beforeAll/afterAll
Global deadline	`SDK_EVAL_MAX_DURATION_MS` + `process.exit(1)`	60 minutes	Cap cumulative cost

Plus one **semantic detector**: the AskQuestion abort pattern in `collectRun` that breaks the event stream loop when the agent requests user input that will never arrive.

The Four Missing Detection Classes

1. **Tool-call loops**: Agent repeatedly calls the same tool with identical arguments. The event stream carries full structured data (`name` , `args` , `status`) to detect this mechanically.
2. **Thinking-without-acting**: Extended `thinking` events with no interleaved `tool_call` events — reasoning loops that burn tokens without producing observable progress.
3. **File-polling deadlocks**: Parent agent waits for child output files that will never appear (because the child has hung). The meditate protocol's polling loop has zero timeout — a single hung leaf blocks the entire tree.
4. **Phantom completions**: Agent reports success, exits cleanly, but produces no artifact on disk. Bypasses all time-based defences. Requires ground-truth verification (filesystem checks, not agent self-report).

The All-or-Nothing Gap

The harness has no intermediate state between "one agent is struggling" and "kill everything." Specifically: - Meditation polls forever for all 3 branch files — no partial-result path - Spec execution phases block until all agents in the phase complete - Shared test runs cascade-fail all N dependent tests when the shared agent crashes - `withRetry` only retries rate-limit errors — all other failures throw immediately

Key Metrics from the Codebase

- Per-test timeouts range from 120s to 480s (2–8 minutes) depending on test complexity
- The meditate protocol spawns up to 39 agents in a full tree (3^0 + 3^1 + 3^2 + 3^3 = 40 total)
- Adversarial verification catches issues at a 25% hit rate (6/24 subtasks across two specs)
- Rate-limit retry uses exponential backoff: 2s base, 60s cap, 5 max retries, ±15% jitter

Connections

Progress Tokens as Unifying Framework

All semantic hang patterns reduce to one question: "Is the agent making forward progress?" Observable progress signals include: new unique tool calls (name+args not seen before), new files written to disk, novel text in the output stream, completed subtask events. **Stagnation** = none of these signals fire within a configurable window. A `lastProgressTimestamp` tracker would subsume tool-call loops, thinking-without-acting, and file-polling deadlocks into a single detection mechanism.

Phantom Completions ↔ Adversarial Verification ↔ Ground Truth Designation

Phantom completions connect to a deeper pattern visible across multiple memories: **when a system has multiple representations of truth (agent narrative, filesystem, spec index, tool-call log), they will silently diverge**. The remedy is always the same: designate one representation as ground truth (filesystem for outputs, subtask details for specs) and automate verification of all others against it.

The harness already has the primitives (`fileExists` , `assertMemoryExists` , `readFile` , `collectRun` 's tool-call capture) but doesn't yet wire them into a systematic "completion verifier" that cross-references tool-call streams against disk state in real time.

Correlation-First Triage for Partial Results

The highest-leverage addition to the failure-handling layer is a **correlation check** as the first triage step: - All branches fail → systemic issue (bad prompt, API down) → escalate immediately, don't retry - One branch fails → isolated issue → retry the leaf or degrade gracefully

This single check prevents the most wasteful failure pattern: retrying a broken prompt N times before the global deadline finally kills everything.

Exploratory vs. Transactional Operations

The decision to proceed with partial results depends on the operation's nature: - **Exploratory** (meditation, recall, research): Partial results are almost always better than nothing. The parent can annotate the gap explicitly. - **Transactional** (spec execution, memory creation): Partial results are dangerous — they may leave the system in an inconsistent state. Escalate to user.

File-Based Coordination Enables Selective Retry

The meditation protocol's file-based coordination already supports a key resilience feature: child outputs persist independently of the parent. If a depth-2 parent hangs while aggregating, it can be re-run and will find existing child files. If a depth-3 leaf fails, only that leaf needs re-running — the siblings' outputs remain on disk. This makes **selective retry at the lowest possible level** feasible without reconstructing state.

Child Subfocuses

Sub-4: Semantic Hang Signals Beyond AskQuestion

Rationale: The single existing semantic detector (AskQuestion abort) covers one hang pattern. The event stream carries enough structured data to detect at least four more: tool-call loops, output stagnation, thinking-without-acting, and file-polling deadlocks. This child explores what detection patterns look like and how to avoid false-positives on legitimately slow agents.

Sub-5: Phantom Completions and Silent Failures

Rationale: Phantom completions are uniquely insidious because they defeat all time-based defences. The agent exits cleanly — no timeout fires, no error thrown. This child explores the taxonomy of phantom subtypes and what a systematic verification layer looks like.

Sub-6: Partial-Result Decision Logic in Fan-Out Failures

Rationale: The current harness is all-or-nothing. This child explores the decision framework for determining retry vs. degrade vs. escalate, including cost awareness, correlation detection, and the graceful consolidation pattern.

Child Insights

From Sub-4 (Semantic Hang Signals)

Five distinct semantic hang patterns were catalogued, each with detection mechanism and false-positive mitigation:

1. **Tool-call loop** — Hash `(name, args)` tuples; 3 consecutive identical = warning, 5 = abort. Discriminate on args identity (not just tool name) to avoid false-positive on legitimate multi-file reads.
2. **Output stagnation** — Sliding-window comparison of the last N characters; cosine similarity >0.9 over 3+ cycles after minimum output length (2000 chars).
3. **Thinking-without-acting** — Ratio of thinking events to tool calls; 10+ consecutive thinking events without a tool call after 60s of elapsed time.
4. **File-polling deadlock** — Track Shell tool calls matching file-existence patterns; same path checked >10 times without success = deadlock. The meditate protocol's unbounded polling is the concrete instance.
5. **Phantom completion** — Post-run cross-reference of Write tool calls against filesystem state. A Write with status "completed" where the target file doesn't exist = phantom.

The **most impactful first build** is the tool-call loop detector (mechanically simple, low false-positive risk, catches common failure mode). The **most urgent gap** is the meditate polling deadlock (zero timeout on file-based coordination means a single hung leaf blocks the entire tree indefinitely).

A unifying **progress token** framework: track `lastProgressTimestamp` based on novel tool calls, new files written, or novel output text. Stagnation = no progress tokens within a configurable window.

From Sub-5 (Phantom Completions)

Four phantom completion subtypes were identified: 1. **Silent Write failure** — Tool returns success but no-ops 2. **Narrative hallucination** — Agent describes file creation without invoking Write 3. **UI-only emission** — Content rendered in chat UI only, never written to disk 4. **Partial Write** — File created but content is truncated/placeholder

A **four-layer verification pyramid** addresses all subtypes at appropriate cost: - Layer 1 (free): Tool-call stream inspection — did a Write even occur? - Layer 2 (cheap): Filesystem existence + content validation - Layer 3 (moderate): Structural output schemas forcing falsifiable claims - Layer 4 (expensive): Adversarial re-verification by independent agent

The deeper principle: **the filesystem is always ground truth; the agent's narrative is always a hypothesis**. This connects to spec index drift (memory d944d7c) and tooling defaults drift (memory 96a7410) — all instances of the same multiple-representations-of-truth problem.

The harness already has Layer 1-2 primitives (`fileExists` , `assertMemoryExists` , `collectRun` tool-call capture) but doesn't yet cross-reference them systematically.

From Sub-6 (Partial-Result Decisions)

A decision matrix was developed: `failure_mode × branch_criticality × correlation → action` : - Correlated failures (all N branches) = systemic → escalate immediately - Isolated failure (1/N) + non-critical branch = proceed with partial results - Isolated failure + critical branch = retry once at lowest possible level

Selective retry cost in the meditation tree: retrying a leaf costs 1 agent; retrying a depth-1 branch costs 13. The file-based coordination already supports this — child files persist, enabling parent re-aggregation without re-running successful children.

The graceful consolidation pattern: Parent writes partial synthesis with explicit acknowledgment of missing branches, impact assessment, and recommendation to re-run the gap. This is strictly better than all-or-nothing for exploratory operations.

Extending `withRetry` : The existing rate-limit-only retry could be generalized to a multi-signal decision: `shouldRetry(failure) = NOT correlated AND type IN [timeout, transient] AND idempotent AND retry_count < max AND cost < threshold` .

Summary

The CRUX harness has a solid time-based defence (3-tier timeout hierarchy) but critical gaps in semantic detection and graceful degradation. Three key findings:

1. **Semantic hang detection needs expansion**: Beyond AskQuestion abort, the event stream supports detecting tool-call loops, thinking-without-acting, file-polling deadlocks, and phantom completions. A unifying "progress token" framework (track last novel action; flag stagnation) would subsume all patterns into a single configurable mechanism.
2. **Phantom completions require a verification layer**: Time-based defences cannot catch an agent that exits cleanly but produces nothing. A four-layer verification pyramid (tool-call stream → filesystem checks → structural schemas → adversarial re-verification) provides defence-in-depth at graduated cost. The filesystem is always ground truth.
3. **Partial-result degradation is missing and needed**: The harness is all-or-nothing at every level. For exploratory operations (meditation), a graceful consolidation pattern — poll with timeout, proceed with available results, annotate the gap — is immediately implementable and strictly better. For transactional operations, correlation-first triage (all fail = systemic, one fails = retry leaf) prevents the most wasteful failure pattern: retrying broken inputs.

The most urgent concrete fix: **add a polling timeout to the meditate protocol's file-based coordination** so a single hung leaf agent cannot block the entire meditation tree indefinitely.

Branch 2, Depth 2, Sub-3: Adversarial Verification as Post-Hoc Failure Detection

Subfocus Rationale

Among the failure-handling strategies available to agent harnesses, adversarial verification occupies a unique niche — it doesn't prevent failures or retry them, it *discovers* them after the fact through independent cross-referencing. With a 25% empirical hit rate across 24 subtasks and documented cases of catching completely invisible failures (files that were never written, extensions that silently changed), adversarial verification demands dedicated analysis of its failure taxonomy, its decision boundaries, and its cost-benefit calculus.

Discoveries

The foundational principle

[memory:Agent-reported file creation must be verified on disk] The most important insight from the memory corpus is epistemological: **an agent's narrative is a hypothesis, not evidence.** Filesystem state is ground truth. An executing agent reported a file as created — with full contents, path, and verification steps in its work log — yet the file never existed on disk. The Write tool either failed silently or the content was emitted only to the chat UI. The agent had no way to know this because it "saw" the content in its own context window.

This inverts the default trust model. Without adversarial verification, the harness trusts the executor's self-report. With it, every claim is treated as falsifiable and checked against independent evidence.

Empirical hit rate: 25% overall, 7-50% variance

[memory:Adversarial verification catches real documentation gaps] Across two plans: - Plan 1 (crux-memories, 14 subtasks): 1 issue found (7%) - Plan 2 (memories-plugin, 10 subtasks): 5 issues found (50%) - Combined: 6 issues in 24 subtasks (25%)

The variance is the key signal. Plan 2 was an integration effort touching config, installer, docs, and distribution — exactly the cross-cutting pattern where failures hide at system boundaries. Plan 1 was mostly isolated skill implementations. The failure rate correlates with the degree of cross-cutting integration, not the number of subtasks.

Failure categories: what adversarial verification excels at

Five distinct categories emerge from the memory corpus, all sharing the structural property of **cross-reference divergence across distributed sources of truth**:

1. **Silent persistence failures** — agent claims file was written; file doesn't exist ([memory:49303e0]). Adversarial advantage: fresh context means no confirmation bias.
2. **Documentation-reality drift** — docs reference `.sh` when actual files are `.py` ([memory:dbfd3ed]). Adversarial advantage: no muscle memory of historical filenames.
3. **Cross-file consistency violations** — spec index contradicts subtask details; command family expansion misses sibling cross-references ([memory:d944d7c]). Adversarial advantage: reads all layers simultaneously rather than relying on sequential recall.
4. **Default/spec misalignment** — tool hardcodes 20% compression target while spec says 25% ([memory:96a7410]). Adversarial advantage: approaches both files without assuming either is correct.
5. **Distribution completeness gaps** — file is in repo but missing from dist zip DIST_FILES ([memory:aba710d]). Adversarial advantage: checks delivery paths the developer workflow never exercises.

Known weaknesses

Adversarial verification produces false positives when it lacks context about deliberate exceptions ([memory:62c0212] — 41 pre-existing SDK TypeScript errors flagged as new regressions; [memory:826c280] — judge demanded regeneration of a transient install artifact). It cannot detect semantic correctness, performance regressions, security vulnerabilities, or end-to-end integration flow failures — all of which require runtime execution rather than cross-reference inspection.

Connections

The cross-reference pattern: All five "excels" categories involve a discrepancy between two independent sources of truth. Adversarial verification is fundamentally a cross-reference checker — it holds two documents side by side and reports divergence. Its structural advantage is that the verifier has no prior commitment to either source being correct.

The false-positive / false-negative tradeoff: A verifier with no historical context catches problems a biased executor would miss — but it also flags non-problems that an informed reviewer would ignore. The memory corpus contains both types: real catches (49303e0, dbfd3ed) and documented false-positive patterns (62c0212, 826c280). This suggests adversarial verification needs a **known exceptions manifest** — analogous to `.eslintignore` — that suppresses previously-triaged baseline noise.

Specialization amplifies verification quality: [memory:Specialized agents outperform generalPurpose] Specialized verifier agents carrying domain-specific checklists would catch more with fewer false positives than general-purpose reviewers. The typed checklist (distribution manifest check, path existence check, cross-reference check) is more effective than open-ended "verify correctness."

Child Subfocuses

Child 1 (depth-3-sub-7): Taxonomy of Failure Categories

Rationale: The aggregate 25% hit rate is uninformative for resource allocation. Mapping specific failure categories to their detection mechanisms reveals where verification effort yields the highest returns and where it's structurally blind.

Child 2 (depth-3-sub-8): Recovery vs Escalation Boundary

Rationale: Catching a failure is only half the problem. The harness must then decide whether to retry silently or present the discrepancy to the user. Getting this boundary wrong in either direction degrades the system — over-escalation fatigues the user; over-recovery masks systematic rot.

Child 3 (depth-3-sub-9): Verification Sampling Strategy

Rationale: Universal verification is wasteful at a 7% hit rate and insufficient at 50%. A risk-targeted sampling strategy grounded in empirical failure categories optimizes the cost-benefit tradeoff.

Child Insights

From depth-3-sub-7: Failure Taxonomy

The taxonomy reveals a clean structural pattern: adversarial verification excels at detecting **cross-reference divergence** — any failure where truth is distributed across multiple files and a single agent works on them sequentially. Five categories (persistence, documentation drift, cross-file consistency, default/spec alignment, distribution completeness) all share this property.

The taxonomy also identifies a critical counter-pattern: **false-positive inflation from missing context**. Pre-existing SDK errors, transient install artifacts, and spec-layer authority rules all produced confident-sounding CRITICAL findings that were categorically wrong. The mitigation is a "known exceptions manifest" — a pre-loaded set of patterns the verifier should not flag.

The structural insight is that self-verification fails because the agent's context window is a single thread (no cross-file comparison), while adversarial verification fails because its fresh context lacks institutional knowledge (no historical exceptions). The ideal system combines both: adversarial checks augmented with a baseline knowledge document.

From depth-3-sub-8: Recovery vs Escalation Framework

The decision boundary decomposes along three axes: 1. **Failure mechanicity**: Mechanistic/tool-level failures (file not persisted) → auto-recover. Judgmental/semantic failures (spec index contradicts subtask) → escalate. 2. **Operation reversibility**: Idempotent operations → auto-recover is safe. Destructive operations → always escalate, even with high confidence. 3. **Verifier confidence**: High confidence (binary check via `ls`) → auto-recover. Low confidence (inferred discrepancy) → escalate.

The critical hidden risk is that auto-recovery **eats failure signals**, masking systemic problems. The mitigation is **advisory auto-recovery**: fix the problem AND log it prominently, with a circuit breaker (retry counter per failure category) that forces escalation when the same failure category recurs. This preserves both execution velocity and diagnostic visibility.

A fourth dimension — **retry cost** — gates even mechanistic failures: cheap retries (file writes, index rebuilds) are auto-recoverable, but expensive retries (multi-agent subtask re-execution) should always escalate because the cost asymmetry flips.

From depth-3-sub-9: Verification Sampling Strategy

A three-tier graduated model optimizes the cost-benefit tradeoff:

Tier	Scope	Cost	Trigger
	ALL subtasks		Default

Tier	Scope	Cost	Trigger
1 — Automated checks		Near-zero (shell commands: `ls`, `git status`, size checks)	
2 — Targeted adversarial	HIGH-RISK subtasks	1 agent invocation	Risk score ≥ 3
3 — Deep specialized audit	CRITICAL subtasks	Specialized verifier agent	Risk score ≥ 5

The risk heuristic assigns points based on empirically-observed failure categories: file creation (+3), cross-cutting registration (+3), documentation with paths (+2), first-time agent assignment (+2), spec/subtask misalignment (+2).

Key cost-reduction strategies: (1) run verifiers in parallel with the next phase's implementation agents — converts verification from a serial bottleneck to parallel overhead; (2) batch related checks into single agent passes using the shared-agent-runs pattern; (3) use advisory gates (`failClosed: false`) so only critical findings block progression.

The core economic insight: verification is a fixed bounded cost (one agent invocation) while undetected failure cost compounds across sessions and is potentially unbounded — making even moderate-hit-rate verification strongly positive expected value when targeted at high-risk subtasks.

Summary

Adversarial verification is the most empirically-grounded failure detection mechanism in the memory corpus, with a 25% hit rate across 24 subtasks and documented catches of completely invisible failures. Its power comes from the **cross-reference divergence** structural pattern — an independent agent with no prior context can check claims against ground truth in ways the executing agent structurally cannot. Five failure categories are well-served (silent persistence, documentation-reality drift, cross-file consistency, default/spec alignment, distribution completeness); semantic correctness, performance, and security are not.

The system's three key design decisions are: (1) **graduated verification** — automated filesystem checks on all subtasks, targeted adversarial review on risk-scored high-value subtasks, deep specialized audits on critical ones; (2) **advisory auto-recovery** — fix mechanistic failures but log them with circuit-breaker escalation, always escalate judgmental failures and destructive operations; (3) **known exceptions manifest** — suppress false positives from pre-existing baseline noise, transient artifacts, and deliberate exceptions. Together, these transform adversarial verification from a uniform tax on all subtasks into a targeted, cost-effective quality amplifier.

Branch 2, Depth 3, Sub-1: Error Classification Heuristics

The current `isRateLimitError()` implementation in `harness.ts` makes a pragmatic, well-documented choice: substring matching on `err.message` with a hardcoded list of 5 phrasings, `instanceof Error` guard, and binary classification (rate-limit → retry, else → throw). This is correct for the current SDK error surface but has three latent fragilities: (1) `"429"` as a bare substring can false-positive on non-rate-limit messages containing that number, (2) `instanceof Error` breaks across serialisation boundaries, and (3) the binary classification discards the "probably transient" category (server errors, network errors) that a production harness would want to retry with a shorter budget. The strongest extensibility direction is toward structured error types with explicit `retryable` properties (as demonstrated by the `IntegrationError` pattern in the sample codebase), with the configurable pattern array as the practical intermediate step. The global timeout (`SDK_EVAL_MAX_DURATION_MS`) provides a critical safety net against classification false positives, forming a defence-in-depth pair with the classifier itself.

Branch 2, Depth 3, Sub-2: Backoff Parameter Tuning and Retry Budget

The CRUX harness backoff parameters are more conservative than their documentation suggests — the actual worst-case delay budget is **62s nominal (53–71s with jitter)**, not the 122s stated in memory `6265f8f` , due to an off-by-one in the delay count. This leaves 75%+ of the 300s test timeout for actual operations. The ±15% jitter is adequate for the current

`maxForks: 2` ceiling but would need widening to ±50% or decorrelated jitter at 4+ forks (collision probability jumps from 10% to 56%). The 2s base delay likely wastes the first 1–2 retry attempts against typical 10–60s rate-limit reset windows; a base of 5s with 4 retries would maintain the same total budget while reducing wasted attempts. The MAX_DELAY cap of 60s is effectively unreachable under current parameters. Most importantly, the shared-agent-runs-per-describe-block pattern is a structural enabler for the narrow jitter — without it, effective concurrency would exceed what ±15% jitter can safely desynchronise.

Branch 2, Depth 3, Sub-3: Pairing Reactive Retries with Proactive Reduction

The CRUX harness implements a four-layer resilience stack (demand reduction → concurrency limiting → reactive retry → hard deadline) where each layer handles what the previous misses. Retries are never vestigial — they serve as both recovery mechanism and feedback signal for tuning proactive measures. The key insight is that reactive and proactive strategies form a feedback loop: retry frequency tells you whether your proactive settings are correct, so removing retries breaks the tuning mechanism even if they rarely fire. The "reduce forks rather than increase retries" guideline generalises to "reduce demand rather than increase tolerance" — a principle that applies to production API clients, multi-agent orchestration, and connection pool management. The interaction between skip-by-default gating and retries creates a bimodal profile where the retry mechanism's importance is inversely proportional to how much proactive gating is in effect. Concurrency is not a single knob but a propagating constraint across test design (shared runs), test selection (skip gates), runtime config (maxForks), and hard limits (wall-clock deadline) — all four must be coherent.

Branch 2, Depth 3, Sub-4: Semantic Hang Signals Beyond AskQuestion

The harness's event stream carries five distinct semantic hang signal categories beyond AskQuestion: **tool-call loops** (same name+args repeated), **output stagnation** (repeated text chunks), **thinking-without-acting** (reasoning events with no tool calls), **file-polling deadlocks** (existence checks that never resolve), and **phantom completion** (claimed outputs that don't exist on disk). Each has a characteristic false-positive risk that requires specific mitigation — args identity for loops, minimum duration for thinking, cycle count for stagnation, and ground-truth verification for phantom completion.

The most impactful detector to build first is the **tool-call loop detector**, because it's mechanically simple (hash comparison on structured data), has low false-positive risk (args identity is a strong discriminator), and catches the most common failure mode. The most urgent gap to close is the **meditate polling deadlock**, because the protocol has zero timeout on file-based coordination — a single hung leaf agent can block the entire meditation tree indefinitely.

A unifying concept — **progress tokens** — would let the harness track a `lastProgressTimestamp` and flag any agent that hasn't demonstrated forward progress

within a configurable window. This subsumes all five patterns into a single framework: an agent is hung when it's alive but producing no progress tokens.

Branch 2, Depth 3, Sub-5: Phantom Completions and Silent Failures

Phantom completions are the most insidious agent failure mode because they bypass all time-based defences and exploit the gap between an agent's self-report and filesystem reality. The CRUX harness already has the primitives for a robust defence — `fileExists`, `assertMemoryExists`, `readFile`, `collectRun`'s tool-call capture, and adversarial judge agents — but these layers are not yet wired into a systematic "completion verifier" that cross-references tool-call streams against disk state.

The key insight is a **four-layer verification pyramid**: (1) tool-call stream inspection (did a Write even occur?), (2) filesystem existence and content checks (did the file materialize with correct content?), (3) structural output schemas (does the agent's response make falsifiable claims we can check?), and (4) adversarial re-verification by a separate agent (does an independent observer confirm the artifacts?). Each layer catches phantom subtypes that lower layers miss, and the cost scales with the layer — making it practical to apply layers 1-2 universally and layers 3-4 selectively.

The deeper principle connecting phantom completions to spec drift (memory d944d7c) and tooling defaults drift (memory 96a7410) is: **when a system has multiple representations of the same truth, silently divergent representations are inevitable; the remedy is to designate ground truth and automate verification of all other representations against it**. For agent output, the filesystem is always ground truth. The agent's narrative is always a hypothesis.

Branch 2, Depth 3, Sub-6: Partial-Result Decision Logic in Fan-Out Failures

The current CRUX harness has no partial-result decision logic — it's all-or-nothing at every level (meditation polls forever; spec phases block on all agents; shared test runs cascade-fail). Three patterns emerge from the analysis:

1. **Correlation-first triage**: Check whether failures are correlated (all branches failing = systemic, abort) vs. isolated (one branch = retry or degrade). This is the highest-leverage addition.
2. **Exploratory vs. transactional split**: Exploratory operations (meditation, recall, research) should degrade gracefully with explicit gap annotations. Transactional operations (spec execution, memory writes) should escalate to the user rather than proceeding with partial results.
3. **Selective retry with cost awareness**: Retry at the lowest possible level (leaf, not subtree). The file-based coordination pattern already supports this — child outputs persist

independently. Extend `withRetry` from rate-limit-only to a multi-signal decision function considering correlation, idempotency, cost, and failure mode.

The graceful consolidation pattern — where the parent explicitly acknowledges missing branches and provides impact assessment — is the most immediately implementable improvement, requiring only a polling timeout and a template for partial synthesis.

Branch 2, Depth 3, Sub-7: Failure Category Taxonomy for Adversarial Verification

Adversarial verification excels at five failure categories, all sharing the structural property of **cross-reference divergence across distributed sources of truth**: (1) silent persistence failures (agent narrative vs. filesystem), (2) documentation-reality drift (docs vs. disk), (3) cross-file consistency violations (spec layers vs. each other), (4) default/spec misalignment (tool code vs. specification), and (5) distribution completeness gaps (integration vs. packaging). In all cases, the verifier's fresh context eliminates the confirmation bias that makes self-verification structurally blind. However, adversarial verification produces significant false positives when it lacks knowledge of deliberate exceptions (pre-existing errors, transient artifacts), and it cannot detect semantic correctness, performance regressions, security vulnerabilities, or integration-flow failures — all of which require runtime execution rather than cross-reference inspection. The optimal design pattern is adversarial verification augmented with a "known exceptions manifest" and a typed verification checklist, not open-ended review.

Branch 2, Depth 3, Sub-8: Recovery vs Escalation Boundary

The boundary between auto-recovery and user escalation after adversarial verification hinges on three axes: **failure mechanicity** (tool-level vs semantic), **operation reversibility** (idempotent vs destructive), and **verifier confidence** (binary check vs inferred judgment).

Auto-recover when all three align favorably: mechanistic failure, idempotent operation, high verifier confidence, cheap retry cost. The canonical example is a silent file-write failure caught by `ls` — just retry.

Always escalate when any axis is unfavorable: judgmental failure (intent-dependent), destructive operation (irreversible), low verifier confidence (inferred rather than observed), or expensive retry (multi-agent re-execution).

The critical hidden risk is that auto-recovery eats failure signals, masking systematic problems behind invisible retries. The mitigation is **advisory auto-recovery**: fix the problem AND log it prominently, with a circuit breaker (retry counter per failure category) that escalates when the same category recurs, preserving both velocity and diagnostic visibility.

Branch 2, Depth 3, Sub-9: Verification Sampling Strategy

Universal adversarial verification is wasteful at a 7% hit rate and insufficient context at a 50% hit rate — the right answer is **graduated, risk-targeted verification**. Three tiers: (1) automated filesystem checks on all subtasks at near-zero cost, (2) targeted adversarial review on subtasks scoring ≥3 on a risk heuristic (file creation, cross-cutting registration, documentation paths, first-time agent assignments), (3) deep specialized audit on subtasks scoring ≥5. The risk heuristic is grounded in five empirically-observed failure categories from the memory corpus. Verification cost is further amortized by running verifiers in parallel with the next phase and batching related checks into single agent passes. The key economic insight: verification is a fixed bounded cost while undetected failure cost compounds across sessions — making targeted verification strongly positive expected value even at moderate hit rates.

---

5. Branch 3: Resource Governance and Bounded Execution

Branch 3 — Depth 1 Synthesis

Subfocus Rationale

Of the three orchestration facets (state coordination, failure handling, resource governance), resource governance addresses the economic and structural constraints that make multi-agent systems viable in practice. Without bounded execution, any sufficiently complex agent harness will eventually produce unbounded cost, unbounded runtime, or unbounded side effects — making the system unsafe to deploy. This facet is orthogonal to the sibling concerns: state coordination handles *what* flows between agents, failure handling addresses *what happens when things break*, and resource governance answers *how much can run, for how long, and at what cost*.

Discoveries

The Memory Corpus Encodes a Mature Governance Stack

Eight memories directly address resource governance, revealing a system that has evolved through empirical pain rather than top-down design:

1. **Cost-tier classification** (`e05030c`): Four natural cost categories emerged from the eval suite — single-turn (1 call), multi-step (1 long call), recursive (13 calls), integration (5+ turns). The 10x+ cost multiplier between cheap and expensive operations drove skip-by-default gating.
2. **Demand reduction over tolerance increase** (`6415c52`): Shared-agent-runs collapse N API calls to 1, achieving 3-6x savings. The system prefers eliminating unnecessary work over adding retry tolerance for excessive work.
3. **Conservative parallelism + reactive recovery** (`6265f8f`): maxForks: 2 as a starting point, exponential backoff with ±15% jitter for rate limits. Never compensate for too-high concurrency with more retries.
4. **Structural parallelism bounds** (`efc4c24`): Per-phase independence derived from dependency graphs eliminates coordination overhead. The bound is structural (problem shape), not environmental (runtime conditions).
5. **Specialization reduces per-task cost** (`e3c5837`): Domain-specific agents use fewer tokens than generalPurpose, compounding savings across large agent trees.
6. **Advisory gates preserve output** (`b0c02ea`): Non-destructive quality checks that warn but never block — progressive enhancement defaults.
7. **Adaptive escalation within hard caps** (`da3d798`): Compression target → escalate aggressiveness → if still exceeds cap → flag for human. Never silently truncate.
8. **Research caching eliminates redundant exploration** (`fc38ec6`): Repeated meditations on similar topics could skip re-exploration via intermediate result caching — an unrealized but well-defined optimization.

Three Principles Unify the Governance Patterns

Across all eight memories, three design principles recur:

Principle 1: Reduce demand before increasing tolerance. The system first eliminates unnecessary work (batching, skip-by-default, specialization), then bounds what remains (caps, deadlines), then tolerates residual transients (retries, backoff). This ordering is critical — applying tolerance to unreduced demand creates a system that appears healthy while accumulating cost.

Principle 2: Static ceilings, negotiable paths. Maximum cost is always predictable at design time (hard caps, recursion depth limits, wall-clock deadlines). But the path to compliance is flexible (adaptive compression, retry with backoff, type demotion chains). The ceiling never negotiates.

Principle 3: Human escalation as universal terminal. When automated resolution would require data loss, the system refuses to decide. Conflicts require user input, manual review replaces truncation, destructive operations require confirmation. The agent's autonomy boundary is drawn at irreversibility.

Connections

The Three-Layer Control Architecture

The governance patterns compose into a layered architecture analogous to TCP congestion control:

Layer	Function	Agent System Equivalent	TCP Analogue
1. Demand reduction	Eliminate unnecessary work before it starts	Batching, skip-by-default, specialization	Connection reuse, compression
2. Admission control	Bound what can run concurrently and for how long	Static caps, phase-based parallelism, deadlines	Congestion window, slow start
3. Reactive recovery	Handle transient failures at validated load	Exponential backoff, jitter, retry budgets	Fast retransmit, AIMD

The codebase has strong implementations of layers 1 and 3 but layer 2 (admission control beyond static caps) is nearly absent — this represents the highest-value gap.

Cost Tiers Map to User Intent, Not Continuous Gradients

Despite four natural cost categories existing, binary gating (cheap/expensive) is optimal because it maps to user decisions: "fast feedback" vs "comprehensive coverage." A 3-tier "moderate" category lacks a natural decision point. The optimal architecture is binary gating for execution control with N-category labeling for cost observability and attribution.

Subagent Inheritance is Cost-Flag Propagation

Session-scope flags (`ba74013`) that fail to propagate to child agents cause silent governance violations. A parent in "amnesia" mode that spawns a cost-uncapped child defeats the parent's resource intent. The agent spawn boundary is the process fork boundary — governance flags must flow across it with the same guarantees as environment variables flow across process boundaries.

The Escalation Ladder is a Composable Primitive

Adaptive escalation (try → measure → tighten → retry → flag) appears in compression, rate-limit recovery, and type demotion. It has a clean five-parameter interface: initial level, step function, ceiling, invariant predicate, and terminal action. Ladders can nest (per-file inside REM sleep), chain (demotion → archival), or run in parallel (independent ladders share no state).

Read-Only Exploration Bounds Side-Effect Risk

The meditate system's read-only-with-opt-in-persistence (`31fec9d`) represents a governance pattern orthogonal to cost: bounding *blast radius* rather than *expenditure*. A 13-agent recursive exploration is expensive but safe because no agent writes persistent state until the user explicitly approves. This separates the "cost" dimension from the "damage" dimension of governance.

Child Subfocuses

1. Cost-Tier Classification and Skip-by-Default Gating

Rationale: Before concurrency can be bounded or deadlines can be set, the system needs a taxonomy of what's cheap and what's expensive, plus a mechanism preventing expensive operations from running accidentally.

Key findings: Four cost dimensions (invocation count, wall-clock, tokens, spawn depth) reduce to two effective axes (invocations × structural complexity). Binary gating maps to user intent while N-category labeling serves observability. The `!== "false"` inversion pattern is maximally conservative — only one exact string enables execution. Five independent discoverability channels compensate for gate invisibility. The full override stack is: config → env → CLI → session → per-call. Blast radius should independently escalate tier classification beyond raw compute cost.

2. Concurrency Control Patterns for Bounded Parallel Execution

Rationale: Concurrency is the primary multiplier of both throughput and rate-limit pressure — the highest-leverage governance parameter.

Key findings: Five static concurrency caps coexist (maxForks, fan-out width, phase parallelism, mutual exclusion, wall-clock). No dynamic controllers exist — the gap between per-request retry and human config editing. Static caps are correct when constraints are structural; dynamic caps add value when constraints are environmental. A two-axis framework (constraint type × cost-of-error) determines the right choice. The scaling protocol is an empirical ratchet: start at 2, increment by 1, full validation, zero retries required, step back on failure. For decorrelation, ±15% jitter works at N=2 but collision probability scales $O(N^2)$, making it insufficient for larger fan-outs. Four proactive patterns are missing: staggered starts, admission queuing, correlated failure detection, and dynamic jitter windows.

3. Hard Resource Caps Composing with Adaptive Escalation

Rationale: When multiple caps are simultaneously active, their composition determines whether the system is coherent or merely a collection of independent limits.

Key findings: The escalation ladder is a five-parameter bounded loop (initial, step, ceiling, invariant, terminal). Caps classify by reversibility of violation: hard-stop (structural impossibility, graceful termination, abort) vs negotiable (iterative degradation, advisory). The ideal composition is the maxMemorySize hybrid: negotiable compliance path wrapped in a hard boundary with human escalation at the terminal. Three multi-cap conflict resolutions exist: refuse-and-flag (strongest preservation), write-partial-and-stop, hard-kill (weakest). Archive-before-write ordering provides accidental crash resilience. The identified gap: wall-

clock deadlines kill without a drain window — a graceful shutdown signal protocol would close the last data-preservation gap.

Child Insights

From Sub-1 (Cost-Tier Classification)

The four cost dimensions reduce to two effective axes because invocation count and structural complexity capture most variance (wall-clock and tokens follow as correlated secondaries). Binary gating is optimal for execution control because users make binary decisions (fast feedback vs comprehensive), not continuous ones. The skip-by-default mechanism uses maximally conservative inversion logic with five independent discoverability channels. Tier placement uses both quantitative thresholds (5x median, >5min) and structural signals (recursion, multi-turn, external deps). Design-time classification suffices because agent operations have deterministic cost profiles. The missing frontier: blast radius as an independent tier escalation signal, and subagent inheritance as cost-flag propagation across spawn boundaries.

From Sub-2 (Concurrency Control)

Concurrency governance decomposes into three complementary patterns operating at different levels: width selection (static for structural constraints, dynamic for environmental), starting point and scaling (maxForks: 2 → empirical ratchet), and failure decorrelation (jitter, staggered starts, admission control). The codebase philosophy is consistent: reduce demand before increasing tolerance. Shared runs, skip-by-default, and conservative maxForks are all demand reduction. Retries and backoff are residual tolerance for validated demand levels. The deepest gap is layer 2 (proactive admission control between demand reduction and reactive recovery). TCP congestion control provides the architectural template: slow start → conservative maxForks, AIMD → backoff with jitter, congestion window → admission semaphore, ECN → file-based rate-limit signals between agents.

From Sub-3 (Hard Caps and Escalation)

Resource caps compose through a layered architecture: negotiable strategies (escalation ladders) inside hard boundaries (absolute caps) with human escalation as the universal terminal. Five design principles emerged: static ceilings make maximum cost predictable, reversibility determines cap classification, archive-before-write preserves data under crashes, file-based coordination provides crash-consistent partial states, and human escalation handles what automation cannot resolve without data loss. The escalation ladder is formally a five-parameter bounded loop with three decision points (pre-gate, invariant, ceiling) — composable via nesting, chaining, or parallelisation. The identified gap: hard-stop mechanisms lack a graceful shutdown signal, killing without drain windows.

Summary

Resource governance in LLM-powered multi-agent systems converges on a **three-layer control architecture**: (1) demand reduction eliminates unnecessary work before it starts (batching, skip-by-default, agent specialization); (2) admission control bounds what can run simultaneously (static caps for structural constraints, dynamic caps for environmental ones, escalation ladders within hard ceilings); (3) reactive recovery handles residual transient failures at validated load (exponential backoff, jitter, retry budgets). Three design principles unify the stack: **reduce demand before increasing tolerance**, **static ceilings with negotiable compliance paths**, and **human escalation as the universal terminal for irreversible decisions**.

The governance system is already mature in layers 1 and 3 but has significant gaps in layer 2 — no dynamic concurrency controllers, no admission queuing, no staggered starts, and no graceful shutdown signals for hard deadlines. The deepest cross-cutting insight is that **cost classification and blast radius are orthogonal governance dimensions**: a 13-agent recursive exploration is expensive but safe (read-only), while a single-agent destructive operation is cheap but dangerous. Both dimensions must independently escalate governance tier classification. The TCP congestion control model provides a proven architectural template for the full three-layer system, including the missing layer-2 components.

> ### Branch 3, Depth 2, Sub-1: Cost-Tier Classification and Skip-by-Default Gating

Subfocus Rationale

Among the three resource governance subfocuses, this one addresses the upstream classification and gatekeeping question: before you can bound concurrency (sub-2) or enforce hard caps (sub-3), you need a taxonomy of what's cheap and what's expensive, and a mechanism that prevents expensive operations from running accidentally. This is the foundation that the other governance patterns build on.

Discoveries

The Empirical Cost Landscape

The CRUX Memories eval suite provides concrete data on four naturally emergent cost categories [memory:Gate expensive SDK evals behind SDK_EVAL_SKIP_EXPENSIVE]:

Category	Invocations	Wall-clock	Examples
Single-turn	1 agent call	30–90s	Recall, Remember, Forget
Multi-step single-turn	1 agent call (longer thinking)	60–120s	Dream, REM Sleep
Recursive	up to 13 agent calls	minutes	Meditate (3 facets × 3 levels)
Integration	5+ sequential turns	minutes	Dream → Recall → Remember → Forget → Amnesia

These categories emerged organically rather than being designed top-down — a strong signal that cost distributions in agent systems are naturally bimodal (cheap cluster vs expensive cluster) rather than continuous.

Four Axes, Two That Matter

Analysis across the memory corpus identifies four dimensions of cost:

1. **Invocation count** — highest variance axis (1× to 13×), most predictive of total cost
2. **Spawn depth** — structural complexity (flat sequential vs recursive tree), determines failure mode severity
3. **Wall-clock time** — partially correlated with invocation count but also encodes a UX signal (developer patience, CI budget)
4. **Token consumption** — modifiable by agent specialization [memory:Specialized agents outperform generalPurpose] and batching [memory:Shared-agent-runs-per-describe-block reduce LLM eval cost], making it a tunable secondary axis

In practice, dimensions 1+2 (invocation count × structural complexity) capture most cost variance. Wall-clock and token consumption are correlated followers. This reduces the taxonomy design problem to a 2-axis classification.

Binary Gating Is Optimal for Execution Control

The CRUX system implements a binary gate (`SDK_EVAL_SKIP_EXPENSIVE !== "false"`) despite observing four empirical categories. This is correct because:

- Tier boundaries should correspond to **user decisions**, not cost gradients — users want either "fast feedback" or "comprehensive coverage", not "medium"
- A 3-tier model introduces an ambiguous "moderate" tier lacking a natural decision point [memory:Plugin design patterns: advisory gates and progressive enhancement defaults]
- The 5x-median threshold lands cleanly in the bimodal gap between clusters
- Finer categories serve **observability** (cost reporting, regression detection, capacity planning) not **gating**

This yields a **2-layer architecture**: binary gating for execution control, N-category labeling for cost attribution.

The Skip-by-Default Mechanism

The canonical pattern uses maximally conservative inversion logic:

```
const skipExpensive = process.env.SDK_EVAL_SKIP_EXPENSIVE !== "false";
```

Only the exact string `"false"` enables execution. Every other state (unset, typo, empty, `"true"`, `"False"`) defaults to skip. The cognitive friction of the triple-negative (`SKIP` × `!==` × `"false"`) is a feature — it forces the developer to pause and acknowledge what they're opting into.

Discoverability is attacked through five independent channels: console output at setup, `.env.example`, README tables, `package.json` scripts, and test file header comments. Each compensates for the others being missed.

Orthogonal Composition of Protection Layers

Per-category gates and global deadlines are independent:

Layer	Type	Protects against
`SDK_EVAL_SKIP_EXPENSIVE`	Binary gate	Known-expensive running by default
`SDK_EVAL_MAX_DURATION_MS`	Continuous deadline	Unexpected cost in any operation

They compose multiplicatively — neither alone is sufficient. This pattern generalizes: gate what you know is expensive, deadline everything as a backstop.

Multi-Layer Override Stack

The full override precedence across the codebase generalizes to:

config default → env-var override → CLI flag override → session-scope override → per-invocation override

Each layer has different persistence: permanent → process-scoped → shell-scoped → session-scoped → call-scoped. The critical failure mode is when overrides don't propagate across agent boundaries — session-scope flags that aren't inherited by subagents silently violate intent [memory:Session-scope command design].

Tier Placement Criteria

Quantitative thresholds (either triggers "expensive"): - Per-invocation cost ≥5x suite median - Wall-clock >5 minutes

Structural signals that predict quantitative thresholds: - Recursive subagent spawning (always expensive — multiplicative cost) - Multi-turn interactive flows (≥5 turns) - External service dependencies (unpredictable latency + cost)

Design-time classification suffices because agent operations have structurally determined cost profiles — recursion depth, fan-out width, and turn count are knowable from the code. Runtime measurement validates but doesn't override structural predictions.

Connections

Adaptive escalation as tier boundary response

The compression system's pattern — try target → check against cap → escalate → flag for human review [memory:maxMemorySize hard cap may force compression beyond target ratio] — maps to how tier boundaries should respond when an operation unexpectedly crosses from cheap to expensive. Advisory warning rather than silent execution or silent failure.

Batching shifts effective tier but gating should use worst-case

Shared-agent-runs collapse N invocations to 1 (3-6x reduction), but this is a harness optimization, not a change to the operation's inherent cost. Tier gating must protect against the unoptimized path executing, while cost reporting can use the optimized actual.

The wall-clock threshold is a UX signal, not a cost signal

">5 minutes" marks where developer feedback loops break, CI budgets strain, and accidental triggers become genuinely disruptive. This is why wall-clock and cost-multiplier are independent "expensive" criteria — they protect against different failure modes.

Missing criterion: blast radius

No existing memory explicitly encodes blast radius as a tier signal, but it's implicit. A REM sleep touching every memory in the corpus has higher blast radius than a single Recall. Blast radius correlates with cost-of-failure and difficulty-of-rollback — it should escalate tier classification independently of raw compute cost.

Subagent inheritance is the agent-world equivalent of env-var propagation

In production harnesses, cost-tier flags must propagate through the agent spawn tree the same way env vars must propagate through process trees. The amnesia inheritance pattern [memory:Session-scope command design] demonstrates both the mechanism (alwaysApply rules) and the failure mode (silent intent violation when inheritance breaks).

Child Subfocuses

1. **Cost tier taxonomy dimensions and optimal tier count** — explores the four axes of cost (invocation count, wall-clock, tokens, spawn depth), their correlations, and why binary gating with N-category observability is optimal over 3-tier or N-tier gating.
2. **Skip-by-default env-var gating patterns and failure modes** — deep-dives into the `!== "false"` inversion logic, the five-channel discoverability strategy, naming conventions, orthogonal composition with deadlines, and generalization to production agent harnesses.
3. **Concrete criteria and heuristics for tier placement of new operations** — examines quantitative thresholds (5x median, 5min wall-clock), qualitative signals (recursion, interactivity, side-effects), design-time vs runtime classification, and how architectural patterns (batching, specialization) shift effective tiers.

Child Insights

Sub-1: Taxonomy Dimensions

The four cost dimensions reduce to two effective axes because they're correlated. Invocation count has the highest variance (1-13x). The CRUX suite's binary gate despite four observed categories confirms that binary gating maps to user intent (fast vs comprehensive) while finer categories serve cost observability. A 3-tier "moderate" tier lacks a natural user decision point. The optimal architecture is binary gating for control + N-category labeling for attribution. Session-scope inheritance affects cost propagation — a "cheap" parent spawning an "expensive" child creates cost escalation the parent's classification didn't predict, arguing for tree-level rather than root-level tier classification.

Sub-2: Gating Mechanics

The `!== "false"` pattern is maximally conservative — only one string activates, every other state defaults safe. The SKIP naming is superior to RUN naming because it describes the default (observable) behavior. Five independent discoverability channels compensate for each other. Package.json scripts serve as the contract between gate mechanisms and CI workflows, encoding opt-in as named aliases. The full generalized override stack is: config → env → CLI → session → per-call. Advisory vs hard gates form a spectrum — catastrophically expensive operations get hard gates (skip entirely), moderately expensive ones might warrant advisory gates (warn but execute) in production harnesses. The pattern generalizes beyond test suites through the subagent inheritance mechanism.

Sub-3: Placement Criteria

The 5x threshold generalizes to bimodal distributions (most agent harnesses) but fails for continuous distributions where percentile-based thresholds work better. Design-time structural classification (recursive? multi-turn? external dependency?) is sufficient because agent operation cost profiles are structurally determined — unlike database queries. Batching and specialization can shift effective tier, but gating should use worst-case unoptimized cost. Retry amplification can temporarily make cheap operations behave expensively, but tier classification should be based on steady-state cost with retry budgets as a separate concern. Blast radius (implicit in side-effect analysis) should independently escalate tier classification beyond raw compute cost.

Summary

Agent harness cost-tier classification converges on a **2-layer architecture**: binary gating for execution control (cheap=default-on, expensive=opt-in-only) with N-category labeling for cost observability. Four dimensions define cost (invocation count, wall-clock, tokens, spawn depth) but reduce to two effective axes (invocations × structural complexity). The **skip-by-default env-var pattern** (`!== "false"`) is maximally conservative — only one exact string enables execution, every other state defaults safe — with five-channel discoverability compensating for the primary failure mode of gate invisibility. **Tier placement** uses both quantitative thresholds (5x median cost, >5min wall-clock) and structural signals (recursion, multi-turn, external dependencies), with design-time classification sufficient because agent operations have deterministic cost profiles. The gating mechanism composes orthogonally with global deadlines — gates prevent known-expensive defaults, deadlines catch unexpected expense. When generalized to production harnesses, the critical extension is **subagent inheritance**: cost-tier flags must propagate through the agent spawn tree or silently violate intent, just as env vars must propagate through process boundaries. The missing frontier is **blast radius** as an independent tier escalation signal beyond raw compute cost.

Branch 3, Depth 2, Sub-2: Concurrency Control Patterns

Subfocus Rationale

The parent facet covers resource governance broadly — cost tiers, size limits, deadlines, adaptive escalation. This branch narrows to the specific question of how many agents should run simultaneously and what controls prevent that number from causing systemic failures. Concurrency is the primary multiplier of both throughput and rate-limit pressure in agent harnesses, making it the highest-leverage governance parameter to get right.

Discoveries

Five static concurrency caps coexist in this codebase

The repository uses five distinct static caps, each justified by a different constraint type:

Location	Cap	Constraint Type
`evals/sdk/vitest.config.ts`	`maxForks: 2`	Environmental — API rate limits
Meditate agent definition	3-way fan-out per recursion level	Structural — topic space partitioning
Spec executor	2-3 agents per phase	Structural — dependency graph
`deploy-pages.yml`	1 (mutual exclusion)	Structural — deployment serialisation
`SDK_EVAL_MAX_DURATION_MS`	60-minute wall-clock cap	Budget — cost ceiling regardless of parallelism

No dynamic concurrency controllers (AIMD, token bucket, adaptive semaphore) exist. The closest adaptive pattern is the human-in-the-loop scaling advice: "scale maxForks to 3-4 only after a full validation run shows zero rate-limit retries."

The retry layer and the concurrency layer are distinct control loops

Memory `6265f8f` pairs per-request exponential backoff with a suite-level static `maxForks`. These operate at different timescales: backoff reacts to individual failures in seconds; concurrency caps should respond to aggregate pressure across minutes. The codebase currently has no controller between these two layers — the gap where a dynamic concurrency adjuster would sit.

The memory corpus encodes a consistent design philosophy: reduce demand before increasing tolerance

- `6415c52` : Shared agent runs collapse N API calls to 1 (demand reduction)
- `e05030c` : Skip-by-default gates exclude expensive tests (demand reduction)
- `6265f8f` : Conservative `maxForks: 2` paired with retry as last resort (demand reduction + residual tolerance)
- `efc4c24` : Phase-based parallelism bounded by dependency graph (demand bounded by structure)

Tolerance mechanisms (retries, backoff, jitter) handle residual transient failures at a validated demand level. They never substitute for finding the right demand level.

Connections

The adaptive compression pattern is a concurrency controller template

Memory `da3d798` describes adaptive compression escalation: try at target → measure outcome → if constraint violated, tighten by 10pp → retry → flag for human if max tightening still fails. This is the AIMD "decrease" half applied to file size rather than concurrency. The same pattern transplants directly to agent pools: run at concurrency N → if rate-limit rate exceeds threshold, reduce to N-1 → if zero rate limits at N for a full validation run, try N+1.

Static caps are correct when constraints are structural; dynamic caps add value when constraints are environmental

The three-way meditate fan-out is static because the constraint is information-theoretic (three facets partition the topic space). The spec executor's phase parallelism is static because it's dependency-graph bounded. Both derive from problem shape, not runtime conditions. A dynamic controller would add complexity with no benefit.

In contrast, `maxForks: 2` for the eval harness is static only because the dynamic version hasn't been built yet. The constraint (API rate limits) is stochastic, variable, and external. This is exactly the regime where dynamic control would help.

The empirical ratchet protocol is well-defined but manual

The scaling protocol encoded in memory `6265f8f` is: start at 2 → run full validation → require zero retries → increment by 1 → repeat. Failure at any level means step back, never compensate with more retries. This is the correct logic for a dynamic controller, just executed by humans editing config files instead of by runtime code.

Child Subfocuses

Three narrower threads were explored at depth 3:

1. **Static vs dynamic fan-out width caps** (sub-4): Decision framework for choosing between fixed and adaptive concurrency. Key finding: a two-axis framework (structural vs environmental constraint × cost of error) determines when static caps suffice and when dynamic control is worth its complexity.
2. **Conservative starting parallelism as a design pattern** (sub-5): Why `maxForks: 2` is the universal starting point and what the empirical scaling protocol looks like. Key finding: 2 is the minimum concurrency that exercises real concurrent behaviour while maintaining diagnostic clarity. The scaling protocol is an observation-gated ratchet: increment by 1, full validation pass, zero retries required, step back on failure.
3. **Thundering-herd avoidance in bounded-width agent pools** (sub-6): How jitter, staggered starts, and admission control prevent correlated failures. Key finding: the existing ±15% jitter works for N=2 forks but collision probability scales as $O(N^2)$ (birthday problem), making it insufficient for Meditate's 9-concurrent-agent case. Four proactive patterns are absent: staggered starts, admission queuing with slow-start, correlated failure detection, and dynamic jitter windows.

Child Insights

Sub-4: Static vs Dynamic Fan-Out Width Caps

A two-axis decision framework emerged:

	Structural Constraint	Environmental Constraint
Low cost of error	Static, derived from problem shape (meditate 3-way)	Static conservative floor + manual tuning (maxForks: 2)
High cost of error	Static, derived from dependency graph (spec phases)	Dynamic controller (AIMD / token bucket)

The adaptive compression escalation pattern (`da3d798`) provides the template for dynamic concurrency: attempt → measure → escalate → retry → flag-for-human. Session-scope inheritance (`ba74013`) creates a hidden static cap — mutable session state effectively serialises execution unless the harness snapshots state at spawn time. The exponential backoff in `withRetry()` operates at the wrong layer for concurrency control; a true dynamic controller would sit between per-request retry and human config editing.

Sub-5: Conservative Starting Parallelism

`maxForks: 2` is correct because: 1. It's the minimum concurrency that exercises real concurrent behaviour (retry paths, jitter, races) 2. It provides diagnostic clarity — rate limits at 2 indicate an upstream quota problem, not a tuning problem 3. It gives a 2× speedup (sufficient for dev iteration) without entering risky territory 4. It establishes the observable baseline that gates all further scaling

The scaling protocol is a ratchet:

```
start at 2 → full validation → zero retries? → increment by 1 → repeat
failure at any level → step back, never compensate with more retries
```

This reflects the codebase-wide principle: reduce demand before increasing tolerance. Retries handle residual transient failures at a validated concurrency level, never substitute for finding the right level. The specific starting number (2) is provider-dependent, but the method (empirical ratchet with full-suite validation gates) is universal.

Sub-6: Thundering-Herd Avoidance

The existing reactive mitigation (exponential backoff + ±15% jitter) works well for `maxForks: 2`, spreading retries across a 9-second window at the delay cap. But four proactive patterns are missing:

1. **Staggered initial starts**: Meditate launches all 3 branch agents simultaneously at each recursion depth. A 500ms inter-launch delay would spread initial API contact across different rate-limit windows at negligible cost to total execution time.
2. **Admission queuing with slow-start**: A semaphore with inter-acquisition spacing and TCP-style slow-start (begin at W=1, probe up to target). The spec executor's dependency-phased approach approximates this but lacks explicit inter-phase spacing.
3. **Correlated failure detection**: When K of N agents hit rate limits within the same window, trigger pool-level backoff. The existing file-based coordination mechanism (agents write markdown files, parents poll) could carry rate-limit signals via `rate-limit-{timestamp}.signal` files — no shared memory needed.
4. **Dynamic jitter windows**: The fixed ±15% works at N=2 but collision probability scales quadratically. A formula like `jitter_range = 0.3 × sqrt(N)` would scale decorrelation with pool size.

The overall pattern space maps to TCP congestion control: slow start → conservative maxForks, AIMD → exponential backoff with jitter, fast retransmit → correlated failure detection, congestion window → admission semaphore, ECN → file-based rate-limit signals.

Summary

Concurrency control in agent harnesses decomposes into three complementary patterns, each operating at a different level:

Pattern 1 — Width selection: Choose between static caps (correct when constraints are structural: dependency graphs, topic partitioning, mutual exclusion) and dynamic caps (valuable when constraints are environmental: rate limits, variable load). The decision framework is a 2×2 matrix of constraint-type × cost-of-error. This codebase currently uses only static caps.

Pattern 2 — Starting point and scaling: Begin at `maxForks: 2` as the minimum concurrency that exercises real concurrent behaviour. Scale via an observation-gated ratchet: increment by 1, full suite validation, zero retries required, step back on any failure. Never compensate for excessive concurrency with more retries — reduce demand before increasing tolerance.

Pattern 3 — Failure decorrelation: Prevent correlated retries from creating self-reinforcing failure cycles. The existing per-request jitter works at N=2 but four proactive techniques are absent — staggered starts, admission queuing, correlated failure detection, and scaling jitter windows. The file-based coordination already used for agent output can serve as the substrate for pool-level rate-limit signalling, and TCP congestion control offers a well-proven template for the full control architecture.

The deepest insight: concurrency control is not a single parameter (`maxForks`) but a three-layer system — demand reduction (shared runs, skip-by-default gating), proactive admission control (width selection, staggered starts, slow-start), and reactive recovery (backoff, jitter, correlated-failure circuit breaking). The codebase has strong implementations of layers 1 and 3 but layer 2 is nearly absent, representing the highest-value gap for future investment.

Branch 3, Depth 2, Sub-3: Hard Resource Caps and Adaptive Escalation

Subfocus Rationale

The parent subfocus asks broadly how agent harnesses manage resources and prevent unbounded growth. This narrowing focuses specifically on the composition problem: when multiple resource caps (size, depth, time) are simultaneously active and each has its own escalation or enforcement strategy, how do they interact without losing data? This is the structural question that underpins whether a multi-cap governance system is coherent or merely a collection of independent limits.

Discoveries

The Escalation Ladder as Universal Pattern

The memory corpus and codebase reveal that resource governance in CRUX-Compress follows a common five-parameter pattern — the **escalation ladder**: initial level, step function, ceiling, invariant, and terminal action. Three concrete instances exist:

- **Compression sizing**: 33% target → reduce by 10pp per iteration → 5% floor → flag manual review
- **Rate-limit backoff**: 0ms → exponential × 2 with jitter → 5 retries / 60s max → throw
- **Memory type demotion**: current type → demotion chain → goal (immune) → flag for review

Each ladder has three structural decision points: a **pre-gate** (should we enter the ladder at all?), an **invariant check** (did this attempt succeed?), and a **ceiling check** (have we exhausted autonomous options?). The ceiling is always a static configuration value, never dynamically computed — this is what makes the maximum cost predictable at design time.

Two Categories of Caps with Distinct Failure Semantics

Resource caps fall into a taxonomy determined by **reversibility of violation**:

Category	Examples	Failure Mode	Data Risk
Hard-stop (structural)	Read-only exploration, no write tools	Impossible to violate	None
Hard-stop (gate)	Destructive op confirmation, recursion depth	Graceful halt or termination	None — agent writes what it has
Hard-stop (kill)	Wall-clock deadline (`process.exit(1)`)	Abrupt termination	In-flight work lost
Negotiable (iterative)	Compression target, retry count	Degrade quality, retry at new level	None if terminal is halt-and-report
Negotiable (advisory)	Plugin quality gates	Warn and preserve output	None

The determining rule: a cap is hard when violation produces irreversible harm; negotiable when degradation is bounded, detectable, and retryable.

The maxMemorySize Hybrid — Gold Standard for Composition

The most instructive composition pattern is `maxMemorySize`: the cap itself is **hard** (a file cannot exceed 1000 lines), but the path to compliance is **negotiable** (adaptive escalation from 33% → 5%). When the negotiable strategy exhausts itself, the system escalates to a hard gate (flag for manual review, refuse to write). The original file is preserved in archive.

This hybrid pattern — negotiable strategy wrapped in a hard boundary with human escalation at the terminal — is the template for how caps should compose: try progressively harder within the negotiable range, then defer to the user rather than losing data.

Multi-Cap Conflict Resolution

Three resolution strategies emerge when caps conflict simultaneously:

1. **Refuse and flag** (compression at floor still exceeds size cap): No write, original preserved, user decides. Strongest data preservation.
2. **Write partial and stop** (recursion depth limit): Agent writes its current state and terminates cleanly. Data is incomplete but available.
3. **Hard kill with no drain** (wall-clock deadline): In-flight work lost, only disk-committed work survives. Weakest data preservation.

The archive-before-write ordering in compression provides accidental resilience against hard kills: because the original is archived before any compressed file is written, a process kill at any point leaves the original recoverable.

Child Subfocuses

Sub-7: Formal Structure of the Escalation Ladder

Rationale: Before caps can compose, the escalation pattern itself needs a precise structural definition — the loop anatomy, decision points, and terminal conditions that make it embeddable and chainable.

Sub-8: Hard-Stop vs Negotiable Cap Taxonomy

Rationale: The failure semantics that follow from classifying a cap as hard vs negotiable determine whether the system can self-recover and what the user experiences — this taxonomy is the prerequisite for any composition strategy.

Sub-9: Data Preservation Invariant Under Multi-Cap Conflicts

Rationale: When multiple caps are active simultaneously and their prescriptions conflict, the "never lose data silently" invariant is most at risk — this subfocus targets the intersection points where caps compete.

Child Insights

From Sub-7 (Escalation Ladder Structure)

The escalation ladder is a **five-parameter bounded loop** with a clean interface (parameterised config in, success-or-terminal-state out). The three decision points — pre-gate, invariant check, ceiling check — are the structural skeleton that all instances share. Key insight: the ceiling is **always static and configuration-driven**, never derived from the current attempt. This prevents the ladder from negotiating with itself about how hard to try, making maximum cost predictable at design time. The pattern is composable: ladders can be **nested** (one per memory file inside a REM sleep loop), **chained** (demotion ladder feeds archival ladder), or **parallelised** (independent ladders share no mutable state). Advisory gates represent the **degenerate case** — zero-rung ladders that immediately surface to the user.

From Sub-8 (Hard vs Negotiable Taxonomy)

The determining factor for cap classification is **reversibility of violation**. Hard-stop caps protect against irreversible harms through mechanisms ranging from structural impossibility (cheapest — the system literally can't violate the cap) to process termination (most expensive — in-flight work is lost). Within the hard category, there's a "gate hardness spectrum": structural impossibility > graceful termination > halt-and-wait > abort. Good harness design moves caps upward on this spectrum. The classification should be made at **definition time** and the failure semantics should be explicit in the interface contract, not emergent from implementation. The `maxMemorySize` hybrid — hard boundary with negotiable compliance path — represents the ideal composition.

From Sub-9 (Data Preservation Under Conflicts)

The "never lose data silently" invariant is well-preserved when cap conflicts have a clear resolution hierarchy, and fails when hard-stop mechanisms can interrupt without a drain window. The **archive-before-write ordering** provides accidental resilience against deadline kills — an emergent property of operation ordering, not explicit design. **File-based coordination** in the meditation system acts as a crash-consistent protocol: agent crashes produce a well-defined partial state (some files exist, others don't), and any parent can resume from whatever arrived. The deepest gap is between **soft stops** (agent notices the limit and flushes state) and **hard stops** (agent is killed externally with no notice). A **signal protocol** — "you have N seconds to flush" before the hard kill — is the missing piece. The Write tool silent failure (memory `49303e0`) represents an orthogonal, infrastructure-level violation that no application-level cap resolution can address.

Summary

Hard resource caps compose with adaptive escalation through a layered architecture: **negotiable strategies** (escalation ladders with iterative aggressiveness increase) operate inside **hard boundaries** (absolute caps that the system cannot exceed), with **human escalation** as the terminal when automated strategies exhaust. The key design principles are:

1. **Static ceilings**: Maximum autonomous cost is always configuration-driven and predictable at design time
2. **Reversibility determines classification**: Caps protecting against irreversible harm must be hard; caps governing quality targets can be negotiable
3. **Archive-before-write ordering**: Preserves data integrity even under hard kills by ensuring the original is safe before any speculative output is attempted
4. **File-based coordination as crash consistency**: Inter-agent communication via the filesystem produces well-defined partial states that survive agent crashes
5. **Human escalation as the universal terminal**: When automated resolution would require data loss, the system refuses to pick a winner and defers to human judgment

The identified gap is the absence of a **graceful shutdown signal** for hard-stop mechanisms — wall-clock deadlines kill processes without giving agents a drain window. Moving from "abort" to "graceful termination" on the gate hardness spectrum would close the last data-preservation gap in multi-cap scenarios.

Branch 3, Depth 3, Sub-1: Cost Tier Taxonomy Dimensions

Four dimensions define agent operation cost: invocation count, wall-clock time, token consumption, and spawn depth. In practice, these reduce to two effective axes (invocation count × structural complexity) because the dimensions are correlated. The CRUX eval suite's empirical data reveals four natural categories but implements a binary gate — and this is correct. Binary gating (cheap/expensive) maps cleanly to user intent (fast feedback vs comprehensive), while finer categories serve observability and cost reporting. A 3-tier model introduces an ambiguous "moderate" tier that lacks a natural user decision point. The optimal architecture is therefore a 2-layer model: binary gating for execution control with N-category labeling for cost attribution. Tier boundaries should be defined by the 5× median cost heuristic and should use advisory escalation (warn on unexpected cost jumps) rather than hard enforcement.

Branch 3, Depth 3, Sub-2: Skip-by-Default Env-Var Gating Patterns

Skip-by-default env-var gating (`!== "false"`) is a maximally conservative pattern where only the exact string `"false"` enables execution; every other state (unset, typo, empty) defaults to

skip. Its effectiveness depends on multi-channel discoverability (console output, `.env.example`, README, package.json scripts, source comments) because the primary failure mode is not misconfiguration but invisibility — engineers not knowing gated paths exist. The pattern composes orthogonally with global deadlines (per-category binary gates prevent known-expensive operations; continuous time limits catch unexpected expense). Package.json scripts serve as the contract layer between gate mechanisms and CI workflows, encoding opt-in as named aliases that inline the env var. When generalized to production agent harnesses, the pattern extends to a multi-layer override stack (config → env → CLI → session → per-call) with subagent inheritance as the critical propagation mechanism — flags that don't cross agent boundaries silently violate intent, just as env vars that don't cross process boundaries silently skip tests.

Branch 3, Depth 3, Sub-3: Tier Placement Criteria and Heuristics

The CRUX system's tier placement criteria are:

Quantitative (either triggers "expensive"): - Per-invocation cost ≥5x suite median - Wall-clock >5 minutes

Qualitative (structural signals that predict quantitative thresholds): - Recursive subagent spawning (always expensive — multiplicative cost) - Multi-turn interactive flows (≥5 turns) - External service dependencies (unpredictable latency + cost)

The 5x threshold generalizes to systems with bimodal cost distributions (most agent harnesses, since simple vs compound operations naturally cluster). It generalizes poorly to continuous distributions — use percentile thresholds instead.

Design-time placement is sufficient because agent operations have structurally-determined cost profiles (recursion depth, fan-out width, turn count are all knowable from the code). Runtime measurement validates but doesn't override structural classification. This is unlike database-style operations where identical code can have wildly different costs based on data.

Architectural patterns can shift effective tier (batching, specialization, caching), but tier *gating* should be based on worst-case unoptimized cost to protect against the unoptimized path executing.

Branch 3, Depth 3, Sub-4: Static vs Dynamic Fan-Out Width Caps

This codebase uses exclusively static concurrency caps, but for fundamentally different reasons: structural caps (meditate's 3-way fan-out, spec executor's phase parallelism) are correct because the problem shape determines the number; conservative-floor caps (`maxForks: 2`) are pragmatic but suboptimal, compensating for the absence of dynamic control with retry-layer backoff. The adaptive compression pattern in `crux-skill-memory-compress` provides a template for what a dynamic concurrency controller would look like: attempt → measure → escalate → retry → flag-for-human. The key decision framework is: static caps are appropriate when the constraint is structural (problem-derived) or the cost of

suboptimal throughput is low; dynamic caps are justified when the constraint is environmental (rate limits, variable load) AND the cost of getting the cap wrong is high enough to justify the implementation complexity.

Branch 3, Depth 3, Sub-5: Conservative Starting Parallelism

`maxForks: 2` is the universal starting point because it's the minimum concurrency level that exercises real concurrent behavior (retry paths, jitter effectiveness, race conditions) while maintaining diagnostic clarity — if rate limits occur at 2, the problem is upstream quotas, not your architecture. The scaling protocol is an observation-gated ratchet: increment by 1, run a full validation pass demanding zero retries, step back on any failure. The philosophical commitment is "reduce demand before increasing tolerance" — retries compensate for residual transient failures at a validated concurrency level, never substitute for finding the right concurrency level. The number 2 is empirically optimal for typical LLM API rate-limit windows, but the *method* (empirical ratchet with full-suite validation gates) is the universal contribution.

Branch 3, Depth 3, Sub-6: Thundering-Herd Avoidance

The codebase has a solid reactive thundering-herd mitigation (exponential backoff + ±15% jitter in `harness.ts`) that works well for the current `maxForks: 2` case. The jitter window spreads retries across 9 seconds at the delay cap, making collision unlikely for 2 forks. However, four proactive patterns are absent: (1) staggered initial starts for fan-out launches, (2) admission queuing with inter-launch spacing and slow-start, (3) correlated failure detection triggering pool-level backoff, and (4) dynamic jitter windows that scale with pool size. These gaps are most acute in the Meditate command's 13-agent tree (up to 9 concurrent at peak), where the birthday-problem dynamic makes retry collisions near-certain with the current fixed jitter window. The existing file-based coordination mechanism provides a natural substrate for pool-level rate-limit signalling without requiring shared memory. The overall pattern space maps closely to TCP congestion control — slow start, AIMD, fast retransmit, and explicit congestion notification — suggesting that the networking community's decades of work on this problem offers directly applicable solutions.

Branch 3, Depth 3, Sub-7: Formal Structure of the Escalation Ladder

The escalation ladder is a five-parameter bounded loop (initial level, step function, ceiling, invariant, terminal action) with three decision points (pre-gate, invariant check, ceiling check). The system decides to stop escalating and surface to the user when the ceiling is reached and the invariant is still violated — this is always a conjunction of a static bound and a predicate failure, never a dynamic judgment. The terminal action splits into halt-and-report (for data-risk operations) and propagate-and-fail (for side-effect-free operations). The pattern's composability comes from its clean interface: parameterised config in, success-or-terminal-

state out — enabling nesting, chaining, and parallel execution. The key structural property that makes it safe is the static ceiling: the maximum autonomous cost is always known at configuration time, never negotiated at runtime.

Branch 3, Depth 3, Sub-8: Hard-Stop vs Negotiable Cap Taxonomy

Hard-stop caps protect against irreversible harms (data loss, unbounded cost, unauthorized state mutation) through mechanisms ranging from structural impossibility to process termination. Negotiable caps protect quality targets through iterative degradation-and-retry loops that always terminate (either successfully or by escalating to a hard gate like "flag for manual review"). The determining factor is reversibility: if violating a boundary produces harm that cannot be automatically undone, the cap must be hard. The `maxMemorySize` hybrid pattern — hard boundary with negotiable compliance path — represents the ideal composition: the system tries progressively harder to fit, but when all strategies fail, it escalates to the human rather than losing data. This taxonomy directly informs harness design: caps should be classified at definition time, and their failure semantics should be explicit in the interface contract, not emergent from implementation details.

Branch 3, Depth 3, Sub-9: Data Preservation Under Multi-Cap Conflicts

The "never lose data silently" invariant is well-preserved when cap conflicts have a clear resolution hierarchy: the compression skill's escalation ladder with terminal manual-review flag is the gold standard. It fails gracefully when each cap allows the agent to notice and react (recursion depth). The invariant is most at risk when hard-stop mechanisms (wall-clock `process.exit(1)`) can interrupt mid-operation without a drain window. The archive-before-write ordering provides accidental resilience, but there is no systematic "flush before kill" signal protocol. The codebase's implicit resolution pattern for unresolvable multi-cap conflicts is human escalation: refuse to pick a winner that would lose data, and ask the user. The Write tool silent failure (memory `49303e0`) represents an orthogonal, infrastructure-level violation that no application-level cap resolution can address — only post-hoc verification catches it.
